Joint Task Offloading and Service Placement for Mobile Edge Computing: An Online Two-Timescale Approach

Xin Li, Xinglin Zhang, *Member, IEEE*, and Tiansheng Huang

***Abstract*—As a new computing paradigm, mobile edge computing (MEC) pushes the centralized cloud resources close to the edge network, which significantly reduces the pressure of the backbone network and meets the requirements of emerging mobile applications. To achieve high performance of the MEC system, it is essential to design efficient task offloading and service placement schemes, which are responsible for offloading tasks to the edge servers while considering the heterogeneity and diversity of computation services. Our MEC system aims to maximize the long-term average network utility while maintaining the stability of the edge network. Considering that synchronous manner overlooks the scenarios endowed with asymmetric update frequencies for service placement and task offloading, we propose an online algorithm based on the two-timescale Lyapunov optimization in a stochastic network environment without requiring the future information. By making asynchronous decisions on service placement and task offloading with different control parameters V, we can achieve a time-average sub-optimal solution that is close to the offline optimum. In addition, we introduce the varying control parameter $V(t)$ and Ω-additive approximation to enhance the robustness of the proposed algorithm within an error Ω. Finally, rigorous theoretical analysis and extensive trace-driven experimental results show that the proposed algorithm achieves the $[O(1/V), O(V)]$ performance-backlog tradeoff and is more competitive than benchmarks.**

***Index Terms*—Mobile edge computing, service placement, task offloading, two-timescale lyapunov optimization.**

I. Introduction

With the widespread adoption of smart mobile devices and the rapid development of the Internet of Things, a variety of service-demanding and latency-sensitive applications have emerged, such as face recognition, augmented reality (AR), and real-time navigation. These applications generate a significant amount of data-oriented tasks that need to be processed. However, the increasing data growth and the congestion of the backbone network/unstable network latency result in challenges for mobile cloud computing, which relies on offloading tasks to the remote central cloud. To overcome these limitations and ensure quality of service for end devices (EDs) like mobile phones and laptops, a new computing paradigm called mobile edge computing (MEC) [2], [3], [4], [5] has emerged. MEC pushes computing resources, services, and data away from the centralized cloud to the edge network, thereby alleviating the pressure on the backbone network and improving the overall service quality.

MEC systems typically consist of base stations (BSs) equipped with MEC servers that provide computation resources and storage capacities. These systems offer various network functions and computing services to EDs through virtualization technologies like Network Function Virtualization (NFV) [6], [7]. *Task offloading*, which determines how to offload task demands from EDs to the edge network, plays a crucial role in efficiently utilizing these resources in MEC. Consequently, it has become a central research focus in MEC, attracting significant attention [8], [9], [10], [11], [12], [13], [14], [15], [16], [17]. However, existing works often overlook the heterogeneity and diversity of computation services available at different BSs. Unlike the centralized cloud, which possesses abundant computation resources and extensive storage space, resource-limited BSs cannot host all available services. Thus, determining which service to place at which BS for a certain duration becomes a critical issue. This process, known as *service placement* (or service caching), refers to downloading and deploying service-related libraries and databases at BSs in the edge network, enabling the execution of task demands requiring these services and reducing latency while improving service quality [2].

In practice, computation services exhibit heterogeneity in terms of resource requirements. For example, the CPU and storage requirements of an AR service differ from those of a mobile gaming service. Additionally, task demands from EDs are spatially and temporally varying and are often modeled as stochastic processes. In this stochastic network environment, long-term network utility becomes a more relevant performance metric than immediate short-term performance [18]. However, the network resource constraint introduces temporal coupling in the computation service placement decisions, making it challenging to achieve optimal adaptivity without future knowledge of demands.

With the advent of 5G networks, the density of BSs in the edge network has significantly increased, enabling overlapping coverage regions where each user can be covered by multiple

Manuscript received 26 October 2022; revised 6 June 2023; accepted 1 September 2023. Date of publication 5 September 2023; date of current version 6 December 2023. This work was supported in part by the National Natural Science Foundations of China under Grants 62372185 and 61872149, and in part by Guangdong Regional Joint Funds for Basic and Applied Research under Grant 2021B1515120078. Recommended for acceptance by D.M. Blough. *(Corresponding author: Xinglin Zhang.)*

The authors are with the School of Computer Science and Engineering, South China University of Technology, Guangzhou, Guangdong 510006, China (e-mail: lixin_forget@163.com; zhxlinse@gmail.com; cs_tsh@163.com).

Digital Object Identifier 10.1109/TCC.2023.3312283

2168-7161 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:05:51 UTC from IEEE Xplore. Restrictions apply.

BSs. This dense BS deployment leads to a highly coupled environment that further complicates service placement and task offloading strategies, requiring coordination among all BSs [19].

Furthermore, the choice of service placement at each BS affects task offloading decisions, which in turn impact the overall performance of the MEC system. Consequently, recent research has explored solutions that consider joint service placement and task offloading [17], [20], [21]. In these approaches, decisions for service placement and task offloading are made synchronously at each time slot (e.g., second or minute). However, this synchronous manner overlooks scenarios where service placement and task offloading exhibit asymmetric update frequencies. For example, if the time slot is short (e.g., a second), frequently updated services can lead to excessive system costs. Conversely, if the time slot is long (e.g., a minute), a large number of task demands can accumulate in the edge network, resulting in congested backbone network traffic and delayed service responses.

To address these issues, we investigate a novel joint service placement and task offloading problem using a two-timescale framework. This framework generalizes existing synchronous solutions by employing different timescales for making service placement and task offloading decisions, respectively. By doing so, we aim to maintain an acceptable average computation time while considering the long-term network environment and ensuring the stability of the entire edge network. To tackle this challenging problem, we design an online algorithm called O2TL, leveraging Lyapunov optimization [22]. The proposed algorithm can make decisions based on current information of the MEC system without requiring future information.

In brief, we have made the following contributions in this paper:

1) We formulate an online service placement and task offloading problem in a stochastic MEC network, which aims to maximize the network utility while maintaining the stability of the entire edge network.

2) We propose an online algorithm based on two-timescale Lyapunov optimization with the fixed control parameters V called O2TL to efficiently solve the formulated problem without requiring the future information of the MEC system, which achieves the $[O(1/V), O(V)]$ performance-backlog trade-off.

3) We extend the fixed V to a variable parameter $V(t)$ that gradually increases with time, which ensures the stability of the entire edge network while simultaneously achieving the optimal utility, making it a realistic and practical solution. Moreover, we introduce the Ω-additive approximation to enhance the robustness within an error Ω.

4) We evaluate the proposed algorithm through theoretical and numerical analysis, showing that our algorithm achieves time-average close-to-optimal performance and that our algorithm is superior to several benchmarks.

The rest of the paper is organized as follows. Section II summarizes the related work. Section III demonstrates the MEC system model. Section IV proposes the online two-timescale Lyapunov algorithm and analyzes its performance. In Section V, we evaluate the proposed algorithm and discuss the results. Finally, we discuss the future work in Section VI and draw the conclusion in Section VII.

II. Related Work

A. Task Offloading

With the development of MEC platforms [28], [29], [30], task offloading (or computation offloading) becomes a key issue that influences the efficiency of MEC. Chen et al. [8] investigate the multi-user computation offloading problem for mobile-edge cloud computing and propose a distributed computation offloading algorithm by taking advantage of game theory. Considering the computation offloading problem for energy harvesting devices, Mao et al. [9] develop an effective computation offloading strategy to minimize the execution cost in a green MEC system. Chen et al. [11] study the task offloading problem in an ultra-dense network. The formulated problem is a mixed integer nonlinear programming problem, with the aim to minimize the delay while reducing the energy consumption of the user's equipment. Huang et al. [13] focus on the binary computation offloading policy in wireless powered MEC networks and propose an online offloading algorithm based on deep reinforcement learning to maximize the number of processed bits within a unit time. Similarly, a wireless powered multi-user MEC system proposed by Wang et al. [14] aims to minimize the total energy consumption, given that each user has a computation latency constraint. In a MEC system combined with the Internet of Vehicles, Chen et al. [31] propose a multihop task offloading decision model for task execution efficiency, considering the practical need of a "best effort" task with a small delay and the scenario without a MEC server and other infrastructure to assist task calculation.

B. Service Placement

However, the aforementioned works often ignore the heterogeneity and diversity of computation services in the edge network. Thus, researchers also investigate making efficient service placement decisions. Hou et al. [32] first propose a tractable online service replacement algorithm that configures edge-clouds dynamically based on the system history, aiming to minimize the total cost of forwarding requests and downloading new services. With the similar objective, Xie et al. [33] study a dynamic service caching algorithm which can be used to relocate the service caching of BSs and re-associate users with BSs when the services required by users change frequently. Yu et al. [34] study a collaborative service placement problem in MEC and aim to minimize the traffic load caused by service request forwarding. An efficient decentralized algorithm taking advantage of the matching theory is proposed to solve this problem. Chen et al. [35] propose a decentralized algorithm based on parallel Gibbs sampling for the collaborative service placement problem in dense small cell networks with the aim of utility maximization. Pasteris et al. [36] consider a MEC system with heterogeneous service storage capacities and rewards and propose an approximation algorithm with a constant approximation ratio, which performs very closely to the optimum.

Similarly, in MEC-enabled IoV, Wang et al. [36] investigate the microservice-oriented service placement mechanism for resource optimization and latency guarantees, which considers the

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:05:51 UTC from IEEE Xplore. Restrictions apply.

TABLE I
DIFFERENCES BETWEEN OUR SCHEME AND THE MOST RELEVANT WORKS

Works	*Task Offloading*	*Service Placement*	*Resource Allocation*	*Task Migration*	*Task Queuing Stability*	*Asynchronous Scheme*
[20]	✔	✔	✔	✗	✗	✗
[23]	✔	✔	✔	✗	✗	✗
[24]	✔	✔	✔	✗	✔	✗
[25]	✔	✗	✔	✗	✗	✗
[26]	✔	✗	✗	✗	✗	✗
[27]	✔	✔	✔	✗	✗	✗
Our	✔	✔	✔	✔	✔	✔

time-varying network load and service shareability to reduce resource consumption and guarantee the service latency generated by the service placement.

C. Joint Task Offloading and Service Placement

Recently, researchers have begun to study the offline task offloading problem with service placement in MEC, which is more complete but more complex. Xu et al. [20] proposes a joint optimization method for dynamic computation offloading and service caching. In each time slot, service caching and computation offloading decisions will be made synchronously to minimize the computation latency while keeping the total computation energy consumption low. Similarly, Poularakis et al. [21] study an online joint optimization problem of service placement and request routing in MEC networks with storage, computation and communication constraints, and linear relaxation and random rounding are used to design the algorithm to maximize the number of requests served by BSs. Tran et al. [23] also study service cache update and task offloading. An offloading cost model is proposed: when the task requests reach the BSs, the BSs can either download the requested service from the cloud or offload the tasks to the cloud. However, it assumes an unlimited available cache size of BSs. Considering an edge-cloud cooperative task offloading, Fan et al. [24] study the service placement, task scheduling, and resource allocation scheme by including both the edge-edge and edge-cloud cooperation, aiming to minimize the total task processing delay while guaranteeing the long-term stability of all the task queues.

However, few studies take into account the shortcoming using the time-slot synchronous manner in a dynamic scenario. In this work, we investigate a two-timescale framework to solve the joint optimization of online service placement and task offloading by leveraging Lyapunov optimization. In Table I, we summarize the difference between our scheme and most relevant works.

III. SYSTEM MODEL AND PROBLEM FORMULATION

A. System Model

To study the long-term performance of the edge network, we consider a MEC system that operates in a time-slot manner [20], [37]. For clarity, the main notations are summarized in Table II. As illustrated in Fig. 1, the timeline is divided into $K \in \mathbb{N}$ *Time Frames* (coarse-grained, e.g., hours/days), each of which can be regarded as the composition of *Time Slots* (fine-grained, e.g., seconds/minutes) with length $T \in \mathbb{N}^+$. Given a sequence of time slots $\tau \in \mathcal{T} = \{0, 1, \ldots, KT-1\}$, we define $t = kT$ $(k = 0, 1, \ldots)$ as the beginning of each time frame $[t, t+T-1]$.

TABLE II
MAIN NOTATIONS

Notation	Description
t	Index of time frame
τ	Index of time slot
$\mathcal{N}$	Set of base stations (BSs)
$\mathcal{I}$	Set of disjoint individual subareas
$\mathcal{N}_i$	Set of BSs covering subarea $i \in \mathcal{I}$
$\mathcal{I}_n$	Set of subareas covered by BS $n \in \mathcal{N}$
F_n	Maximum CPU frequency of BS n
R_n	Storage capacity of BS n
$\mathcal{S}$	Set of computing services
r_s	Storage space required by service s
f_s	CPU cycles required per bit for service s
a_s	Revenue obtained after completing per bit for service s
$c_{s,n}$	Service placement cost of deploying service s on BS n
$c_{f,n,s}$	Task computation cost on BS per bit for service s
$c_{f,l,s}$	Task computation cost on cloud per bit for service s
$c_{n,l}$	BS-Cloud task migration cost per bit from BS n to the cloud
$d^{\tau}_{s,i}$	Task computation demands (bits) for service s of subarea i at time slot τ
$x^{t}_{n,s}$	Binary decision variable to indicate whether service s is deployed at BS n at time frame t
$y^{\tau}_{n,s,i}$	Continuous decision variable to indicate the fraction of task computation demands $d^{\tau}_{s,i}$ offloaded to BS n at time slot τ
$m^{\tau}_{n,l,s}$	Non-negative real valued decision variable to indicate the task computation demands for services s migrated from BS n to the cloud at time slot τ
$\kappa^{\tau}_{n,s}$	Non-negative real valued decision variable to indicate the computation resource allocated to task computation demands for service s on BS n at time slot τ

We consider a MEC system with N base stations (BSs), defined as $\mathcal{N} = \{1, 2, \ldots, N\}$. Each BS $n \in \mathcal{N}$ has a MEC server providing computing services to EDs (e.g., mobile phones, laptops) within its coverage. As shown in Fig. 2, we divide the network area into I disjoint subareas defined as $\mathcal{I} = \{1, 2, \ldots, I\}$. With the dense deployment of BSs, subarea $i \in \mathcal{I}$ can be covered by a set of BSs defined as $\mathcal{N}_i \subseteq \mathcal{N}$. Similarly, the set of subareas that can be covered by BS n is defined as $\mathcal{I}_n \subseteq \mathcal{I}$.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:05:51 UTC from IEEE Xplore. Restrictions apply.

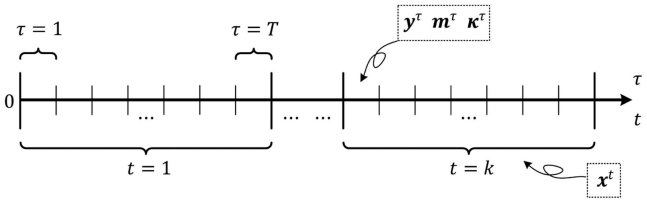


Fig. 1. Time devision with two scales.

BSs provide EDs with different network functions and computing services through some virtualization technologies, such as NFV. We consider that each BS n has limited computation resources to handle computing tasks, defined as F_n (maximum CPU frequency), and limited storage space for service deployment, defined as R_n. Assume that there are S computing services provided by the MEC system, indexed by $\mathcal{S} = \{1, 2, \ldots, S\}$. Each service s requires a storage space r_s to store libraries and databases related to service s, and f_s CPU cycles per bit for computation.

We assume that the task computation demand (bits) for service s of subarea i at time slot τ, defined as $d^{\tau}_{s,i}$, follows an exponential distribution with the parameter $1/\lambda_{s,i}$. The exponential distribution has memoryless property, and thus the expectation of $d^{\tau}_{s,i}$ is $\mathbb{E}[d^{\tau}_{s,i}] = \lambda_{s,i}$. Note that the exponential distribution is commonly utilized in cutting-edge research papers [20], [38]. However, our model is equally applicable to other distributions.

B. Service Placement

Service placement is to download from the cloud server and deploy the libraries and databases related to services at the network edge BSs, thereby reducing latency and improving the quality of services. Placing service s at BS n allows processing tasks requiring service s at the MEC server. However, due to the limited storage space of the BS server, it is impossible to deploy all the services at the same time. On the other hand, downloading related libraries and databases from the cloud server will incur costs, which discourages changing the deployed services very frequently. We assume that the MEC system makes the service placement decision for each BS n at the beginning of each time frame t.

Definition 1. (Service Placement Decision). Let $x^t_{n,s} \in \{0, 1\}$ be a binary decision variable to indicate whether service s is deployed at BS n at time frame t. Here, $\boldsymbol{x}^t_n \triangleq \{x^t_{n,1}, \ldots, x^t_{n,s}, \ldots, x^t_{n,S}\}$ is the service placement decision of BS n for all services at time frame t and $\boldsymbol{x}^t \triangleq \{\boldsymbol{x}^t_1, \ldots, \boldsymbol{x}^t_n, \ldots, \boldsymbol{x}^t_N\}$ are the service placement decisions of all BSs at time frame t. In addition, the storage space occupied by the services deployed at BS n should not exceed the total storage space of BS n at any time frame:

$$\sum_{s \in \mathcal{S}} x^t_{n,s} r_s \leq R_n, \ \forall n \in \mathcal{N}. \tag{1}$$

C. Task Offloading Management

Task offloading management contains a series of operations of the MEC system to handle task computation demands in the network area, which mainly includes three decisions: task offloading decision, task migration decision and computation resource allocation decision of BSs. In a stochastic environment, the task computation demands of EDs are uncertain and constantly changing. To better adapt to the dynamic environment, we assume that the MEC system makes task offloading management decisions at every time slot τ.

1) Task Offloading Decision: Within the dense network, each subarea $i \in \mathcal{I}$ is covered by a set of BSs $\mathcal{N}_i \subseteq \mathcal{N}$. Therefore, the MEC system needs to decide the amount of task computation demands offloaded to each BS.

Definition 2. (Task Offloading Decision). Let $y^{\tau}_{n,s,i} \in [0, 1]$ be a continuous decision variable to indicate the fraction of task computation demands $d^{\tau}_{s,i}$ offloaded to BS n at time slot τ. Here, $\boldsymbol{y}^{\tau}_{n,s} \triangleq \{y^{\tau}_{n,s,1}, \ldots, y^{\tau}_{n,s,i}, \ldots, y^{\tau}_{n,s,I}\}$ is the task offloading decision of BS n for service s at time slot τ, where $y^{\tau}_{n,s,i} = 0$ if $i \notin \mathcal{I}_n$. Moreover, $\boldsymbol{y}^{\tau}_n \triangleq \{\boldsymbol{y}^{\tau}_{n,1}, \ldots, \boldsymbol{y}^{\tau}_{n,s}, \ldots, \boldsymbol{y}^{\tau}_{n,S}\}$ and $\boldsymbol{y}^{\tau} \triangleq \{\boldsymbol{y}^{\tau}_1, \ldots, \boldsymbol{y}^{\tau}_n, \ldots, \boldsymbol{y}^{\tau}_{\mathcal{N}}\}$ denote the task offloading decision of BS n and the task offloading decisions of all BSs at time slot τ, respectively. In addition, the total fraction of task computation demands for services s from subarea i offloaded to BSs in $\mathcal{N}_i$ at any time slot τ is 1:

$$\sum_{n \in \mathcal{N}_i} y^{\tau}_{n,s,i} = 1, \ \forall i \in \mathcal{I}, \forall s \in \mathcal{S}. \tag{2}$$

It can be further calculated that the total task computation demands for service s offloaded to BS n at time slot τ are $D^{\tau}_{n,s} = \sum_{i \in \mathcal{I}_n} y^{\tau}_{n,s,i} d^{\tau}_{s,i}$.

2) Task Migration Decision: As mentioned above, because of the limited resources of BS servers, the BSs and the cloud server (BS-Cloud) can cooperate by migrating task computation demands to the cloud server. Task migration is so important that it can relieve the pressure of the entire edge network and improve the efficiency of task computing.

Definition 3. (Task Migration Decision). Let $m^{\tau}_{n,l,s} \geq 0$ be a non-negative real valued decision variable to indicate the task computation demands (bits) for service s migrated from BS n to the cloud at time slot τ, where l represents the cloud server. Here, $\boldsymbol{m}^{\tau}_{n,l} \triangleq \{m^{\tau}_{n,l,1}, \ldots, m^{\tau}_{n,l,s}, \ldots, m^{\tau}_{n,l,S}\}$ is the task migration decision for all services from BS n to cloud l at time slot τ. Moreover, $\boldsymbol{m}^{\tau} \triangleq \{\boldsymbol{m}^{\tau}_1, \ldots, \boldsymbol{m}^{\tau}_n, \ldots, \boldsymbol{m}^{\tau}_N\}$ denotes the task migration decisions of all BSs at time slot τ.

3) Computation Resource Allocation Decision: Recall that each BS n has limited computation resources F_n. Due to the diverse task computation demands for services at each BS n and the required CPU cycles per bit for service s, it is necessary to allocate computation resources to tasks for different services.

Definition 4. (Computation Resource Allocation Decision). Let $\kappa^{\tau}_{n,s}$ be a non-negative real valued decision variable to indicate the computation resource (CPU frequency, the same as F_n) allocated to task computation demands for service s at BS n at time slot τ. Here, $\boldsymbol{\kappa}^{\tau}_n \triangleq \{\kappa^{\tau}_{n,1}, \ldots, \kappa^{\tau}_{n,s}, \ldots, \kappa^{\tau}_{n,S}\}$ and

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:05:51 UTC from IEEE Xplore. Restrictions apply.

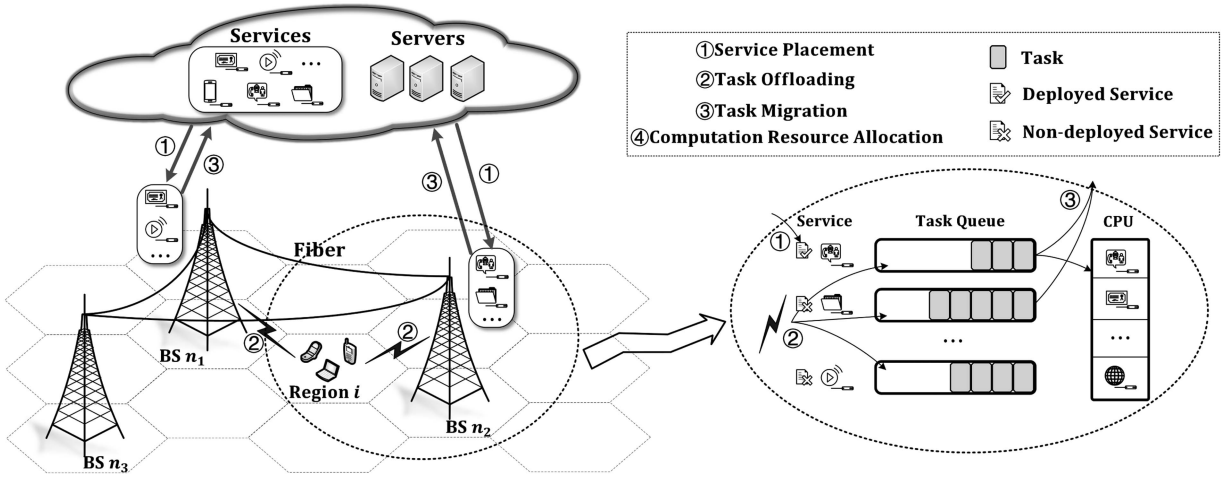


Fig. 2. MEC System.

$\boldsymbol{\kappa}^\tau \triangleq \{\boldsymbol{\kappa}_1^\tau, \ldots, \boldsymbol{\kappa}_n^\tau, \ldots, \boldsymbol{\kappa}_N^\tau\}$ denote the computation resource allocation decisions of BS n and all BSs at time slot τ, respectively. Generally, only when service s is deployed at BS n at time frame t, the computation resource can be allocated to task computation demands for service s at time slot $\tau \in [t, t+T-1]$:

$$\kappa_{n,s}^\tau \in [0, F_n x_{n,s}^t], \forall n \in \mathcal{N}, \forall s \in \mathcal{S}. \tag{3}$$

In addition, the computation resource allocated to tasks for different services at BS n should not exceed the total computation resource of BS n at any time slot:

$$\sum_{s \in \mathcal{S}} \kappa_{n,s}^\tau \leq F_n, \forall n \in \mathcal{N}. \tag{4}$$

D. Task Queuing Model

We assume that each BS maintains a task buffer for each service to store offloaded tasks.

Definition 5. (Task Queuing Model). Let $Q_{n,s}^\tau$ denote the task queue backlog (bits) for service s at BS n at the beginning of time slot τ. Specifically, the initial state $Q_{n,s}^0$ is assumed to be a non-negative real value. Then, the task queue backlog at the next slot $Q_{n,s}^{\tau+1}$ can be calculated as follows:

$$Q_{n,s}^{\tau+1} = \max\left\{Q_{n,s}^\tau - \left(\frac{\kappa_{n,s}^\tau}{f_s} + m_{n,l,s}^\tau\right), 0\right\} + D_{n,s}^\tau. \tag{5}$$

In order to maintain the stability of the entire edge network and each task queue, we need to bound the time average task queue backlogs over time slots:

$$\limsup_{|\mathcal{T}|\to\infty} \frac{1}{|\mathcal{T}|} \sum_{\tau=0}^{|\mathcal{T}|-1} \sum_{n=1}^{N} \sum_{s=1}^{S} \mathbb{E}[Q_{n,s}^\tau] < \varepsilon, \exists \varepsilon \in \mathbb{R}^+, \tag{6}$$

where $\varepsilon \to \infty$, the task queue backlogs are strongly stable. In addition, any task queue backlog $Q_{n,s}^\tau$ for service s at BS n at each time slot τ should not exceed the maximum task queue backlog $Q_{n,s}^{\max}$. We call the task queue backlog is mean rate stable if

$$\lim_{\tau\to\infty} \frac{\mathbb{E}\left[Q_{n,s}^\tau\right]}{\tau} = 0. \tag{7}$$

Based on the definition of the task queue, we can give the constraint on task migration decision variables as follows:

$$\frac{\kappa_{n,s}^\tau}{f_s} + m_{n,l,s}^\tau \in [0, Q_{n,s}^\tau], \forall n \in \mathcal{N}, \forall s \in \mathcal{S}, \forall \tau \in \mathcal{T}, \tag{8}$$

which means that the task migration decision only depends on the known task queue backlog at the beginning of the current time slot, while being independent of the unknown task computation demand.

For convenience, let $\gamma_{n,s}^\tau = \frac{\kappa_{n,s}^\tau}{f_s} + m_{n,l,s}^\tau$ and $\delta_{n,s}^\tau = D_{n,s}^\tau$. Due to the limited bandwidth of wireless channels, we assume that $\gamma_{n,s}^\tau \leq \gamma_{n,s}^{\max}$ and $\delta_{n,s}^\tau \leq \delta_{n,s}^{\max}$.

E. Network Utility and Problem Formulation

Here, we focus on building a long-term stable, green MEC system, for which we need to optimize service placement and task offloading management decisions. We assume that the revenue and cost of BSs are linear to the amount of tasks, which provides a reasonable approximation in many practical scenarios and has been commonly used in literature [20], [23].[1] The specific revenue and cost of the MEC system are defined as follows.

Definition 6. (Revenue). As mentioned above, the MEC system provides S computing services to EDs, and the task computation demands for service s of subarea i at time slot τ is $d_{s,i}^\tau$. Let a_s represent the revenue obtained by the MEC system for completing one bit of the task for service s. Then, the total

[1] Note that our proposed algorithms and analytical framework can be extended to accommodate different cost models if required in future research.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:05:51 UTC from IEEE Xplore. Restrictions apply.

revenue obtained by the MEC system at time slot τ is as follows:

$$A^\tau = \begin{cases} \sum_{n=1}^{N}\sum_{s=1}^{S}\mathbb{E}\left[a_s Q_{n,s}^\tau\right], & Q_{n,s}^\tau < \frac{\kappa_{n,s}^\tau}{f_s}, \\ \sum_{n=1}^{N}\sum_{s=1}^{S}\mathbb{E}\left[a_s\left(\frac{\kappa_{n,s}^\tau}{f_s}+m_{n,l,s}^\tau\right)\right], & Q_{n,s}^\tau \geq \frac{\kappa_{n,s}^\tau}{f_s}. \end{cases} \tag{9}$$

Definition 7. (Service Placement Cost). Let $c_{s,n}$ represent the service placement cost of deploying service s at BS n. Moreover, the service placement cost will be incurred only if service s is not deployed at time frame t but deployed at next time frame $t+T$. Therefore, according to service placement decision $\boldsymbol{x}^t$, the total service placement cost C_p^t of the MEC system at time frame t is as follows:

$$C_p^t = \begin{cases} \sum_{n=1}^{N}\sum_{s=1}^{S}\mathbb{E}\left[c_{s,n}x_{n,s}^t\right], & t=0, \\ \sum_{n=1}^{N}\sum_{s=1}^{S}\mathbb{E}\left[c_{s,n}\left(1-x_{n,s}^{t-T}\right)x_{n,s}^t\right], & t=kT(k\in\mathbb{N}^+). \end{cases} \tag{10}$$

Without loss of generality, we define $C_p^\tau = C_p^t/T$ as the average service placement cost at each time slot $\tau \in [t, t+T-1]$.

Definition 8. (Task Migration Cost). Assume that BSs and the cloud are connected through the backhaul network, thus the task demands can be migrated from BSs to the cloud. On the other hand, the distance from a BS to the cloud is much longer so that if the task demand needs to be offloaded to the cloud, it may need to go through multi-hop migration, which incurs more costs. Let $c_{n,l}$ represents the BS-Cloud task migration cost per bit from BS n to the cloud (transmitted in binary form regardless of services). Then, according to task migration decision $\boldsymbol{m}^\tau$, the total BS-Cloud task migration cost $C_{p,l}^\tau$ of the MEC system at time slot τ is as follows:

$$C_{p,l}^\tau = \sum_{n=1}^{N}\sum_{s=1}^{S}\mathbb{E}\left[c_{n,l}m_{n,l,s}^\tau\right]. \tag{11}$$

Definition 9. (Task Computation Cost). The task computation cost of the BSs and the cloud mainly comes from the energy consumption. Let $c_{f,n,s}$ and $c_{f,l,s}$ represent the task computation cost of BS n and the task computation cost of the cloud per bit for service s, respectively. Then, according to computation resource allocation decision $\kappa_{n,s}^\tau$ and task migration decision $m_{n,l,s}^\tau$, the total task computation cost of the BSs $C_{e,b}^\tau$ and the task computation cost $C_{e,l}^\tau$ of the cloud at time slot τ are as follows:

$$C_{e,b}^\tau = \begin{cases} \sum_{n=1}^{N}\sum_{s=1}^{S}\mathbb{E}\left[c_{f,n,s}\left(Q_{n,s}^\tau - m_{n,l,s}^\tau\right)\right], & Q_{n,s}^\tau < \frac{\kappa_{n,s}^\tau}{f_s}, \\ \sum_{n=1}^{N}\sum_{s=1}^{S}\mathbb{E}\left[c_{f,n,s}\frac{\kappa_{n,s}^\tau}{f_s}\right], & Q_{n,s}^\tau \geq \frac{\kappa_{n,s}^\tau}{f_s}, \end{cases} \tag{12}$$

$$C_{e,l}^\tau = \sum_{n=1}^{N}\sum_{s=1}^{S}\mathbb{E}\left[c_{f,l,s}m_{n,l,s}^\tau\right]. \tag{13}$$

Given that the network utility is the net revenue of the MEC system, defined as $U^\tau \triangleq A^\tau - (C_p^\tau + C_{p,l}^\tau + C_{e,b}^\tau + C_{e,l}^\tau)$, the goal of the MEC system is to maximize the network utility by making service placement and task offloading management decisions while maintaining the stability of the entire edge network and each task queue. Then the studied problem of **O**nline **S**ervice **P**lacement and **T**ask Offloading **M**anagement (**OSPTM**) can be defined as follows:

Definition 10. (OSPTM Problem). The problem of OSPTM is to solve the following optimization problem **P1**:

$$(\mathbf{P1}) \quad \max_{\boldsymbol{x}^t,\boldsymbol{y}^\tau,\boldsymbol{m}^\tau,\boldsymbol{\kappa}^\tau} \limsup_{|\mathcal{T}|\to\infty}\frac{1}{|\mathcal{T}|}\sum_{\tau=0}^{|\mathcal{T}|-1}\mathbb{E}\left[U^\tau\right] \tag{14}$$

$$\text{s.t.} \quad x_{n,s}^t \in \{0,1\},\ \forall n\in\mathcal{N}, \forall s\in\mathcal{S}, \tag{15}$$

$$y_{n,s,i}^\tau \in [0,1],\ \forall n\in\mathcal{N}, \forall s\in\mathcal{S}, \forall i\in\mathcal{I}, \forall \tau\in\mathcal{T}, \tag{16}$$

$$(1)\text{–}(4),\ (6),\ (8).$$

It can be seen that the OSPTM problem is a stochastic optimization problem that maximizes the long-term average network utility. Since the future information (e.g., task computation demand) about the MEC system can not be known in advance, it is very hard to find the optimal solution to maximize the long-term average network utility of the dynamic MEC system. To address this challenge, we use two-timescale Lyapunov optimization [22] in Section IV to convert the original OSPTM problem to the *T*-slot optimization problem, which only requires the information of the current time frame.

Moreover, we define a *feasible* policy $\boldsymbol{\Pi}$ which makes service placement decision $\boldsymbol{x}^t$ at time frame $t=kT$ every T time slots and makes task offloading decision $\boldsymbol{y}^\tau$, task migration decision $\boldsymbol{m}^\tau$ and computation allocation decision $\boldsymbol{\kappa}^\tau$ at every time slot τ subject to the constraints. Then, we define the expectation of the time-average utility achieved by a policy $\boldsymbol{\Pi}$ that meets the constraints as follows:

$$U^{\boldsymbol{\Pi}} = \limsup_{|\mathcal{T}|\to\infty}\frac{1}{|\mathcal{T}|}\sum_{\tau=1}^{|\mathcal{T}|-1}\mathbb{E}\left[U^{\boldsymbol{\Pi},\tau}\right]. \tag{17}$$

Here, $U^{\boldsymbol{\Pi},\tau}$ denotes the network utility achieved under policy $\boldsymbol{\Pi}$ at time slot τ. We use U^{opt} to denote the expectation of the supremum time-average network utility over all feasible policies.

IV. Online Service Placement and Task Offloading Management

In this section, we develop an **O**nline algorithm based on **T**wo-**T**imescale **L**yapunov optimization method (**O2TL**) to solve the OSPTM problem.

A. Two-Timescale Lyapunov Optimization

First, we define $\boldsymbol{\Theta}(t) = [\ldots, Q_{n,s}^t, \ldots]$ as the task queue backlog vector of all BSs for all services at time frame $t=kT$. Next, we introduce the quadratic Lyapunov function $L(\boldsymbol{\Theta}(t))$ as follows, which is a scalar measure of the queue congestion of the MEC network:

$$L(\boldsymbol{\Theta}(t)) \triangleq \frac{1}{2}\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}}(Q_{n,s}^t)^2. \tag{18}$$

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:05:51 UTC from IEEE Xplore. Restrictions apply.

Intuitively, the function $L(\mathbf{\Theta}(t))$ is non-negative. If and only if all $Q_{n,s}^t = 0$, $L(\mathbf{\Theta}(t)) = 0$. Further, if all task queue backlogs are small, the value of $L(\mathbf{\Theta}(t))$ would be small; otherwise, if any task queue backlog is large, the value of $L(\mathbf{\Theta}(t))$ would be large.

Then, we define the T-slot conditional Lyapunov drift as:

$$\Delta_T(\mathbf{\Theta}(t)) \triangleq \mathbb{E}\left[L(\mathbf{\Theta}(t+T)) - L(\mathbf{\Theta}(t))|\mathbf{\Theta}(t)\right]. \tag{19}$$

which measures the expected change in the Lyapunov function over one time frame based on the current task queue backlog vector $\mathbf{\Theta}(t)$. Intuitively, we can maintain the stability of the entire edge network and each task queue by minimizing the T-slot conditional Lyapunov drift.

Inspired by the *drift-plus-penalty* minimization policy in Lyapunov optimization [22], we design a *drift-minus-utility* minimization policy to minimize the upper bound of the following *drift-minus-utility* term every time frame by making service placement decision $\boldsymbol{x}^t$ at each time frame t and task offloading decision $\boldsymbol{y}^\tau$, task migration decision $\boldsymbol{m}^\tau$, computation resource allocation decision $\boldsymbol{\kappa}^\tau$ at each time slot $\tau \in [t, t+T-1]$:

$$\Delta_T\left(\mathbf{\Theta}(t)\right) - V\mathbb{E}\left[\sum_{\tau=t}^{t+T-1} U^\tau|\mathbf{\Theta}(t)\right]. \tag{20}$$

where $V \geq 0$ is a control parameter of tradeoff between task queue backlog minimization and network utility maximization. Specifically, the larger V means the MEC system puts more weight on network utility maximization.

The analytical upper bound of the drift-minus-utility term is given by Theorem IV.1.

Theorem IV.1. (Drift-minus-Utility Bound). Let $V > 0$, $T \geq 1$, $t = kT (k \in \mathbb{N}^+)$. Given that $\gamma_{n,s}^\tau \leq \gamma_{n,s}^{\max}$ and $\delta_{n,s}^\tau \leq \delta_{n,s}^{\max}$, for any feasible police $\mathbf{\Pi}$, the drift-minus-utility is bounded as

$$\begin{aligned}
&\Delta_T\left(\mathbf{\Theta}(t)\right) - V\mathbb{E}\left[\sum_{\tau=t}^{t+T-1} U^\tau|\mathbf{\Theta}(t)\right] \\
\leq &\Gamma_2 T - \mathbb{E}\Bigg[\sum_{\tau=t}^{t+T-1}\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}} Q_{n,s}^t(\gamma_{n,s}^{\mathbf{\Pi},\tau} - \delta_{n,s}^{\mathbf{\Pi},\tau}) \\
&+ V\sum_{\tau=t}^{t+T-1} U^{\mathbf{\Pi},\tau}|\mathbf{\Theta}(t)\Bigg].
\end{aligned} \tag{21}$$

where $\Gamma_2 = \Gamma_1 + \frac{T-1}{2}\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}}(\gamma_{n,s}^{\max})^2$ and $\Gamma_1 = \frac{1}{2}\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}}((\gamma_{n,s}^{\max})^2 + (\delta_{n,s}^{\max})^2)$.

Proof. Based on the task queue backlog (5), $\gamma_{n,s}^\tau$ and $\delta_{n,s}^\tau$, for any time frame t and the time slot $\tau \in [t, t+T-1]$, we have

$$Q_{n,s}^{\tau+1} = \max\left\{Q_{n,s}^\tau - \gamma_{n,s}^\tau, 0\right\} + \delta_{n,s}^\tau. \tag{22}$$

Squaring the task queue backlog (22) for any time slot τ and using the fact that $[\max\{a,0\}]^2 \leq a^2$, $\forall a \geq 0$, we obtain

$$\begin{aligned}
(Q_{n,s}^{\tau+1})^2 = &\left[\max\left\{Q_{n,s}^\tau - \gamma_{n,s}^\tau, 0\right\}\right]^2 + (\delta_{n,s}^\tau)^2 \\
&+ 2\delta_{n,s}^\tau \max\left\{Q_{n,s}^\tau - \gamma_{n,s}^\tau, 0\right\} \\
\leq &(Q_{n,s}^\tau)^2 + (\gamma_{n,s}^\tau)^2 - 2Q_{n,s}^\tau\gamma_{n,s}^\tau + (\delta_{n,s}^\tau)^2 \\
&+ 2\delta_{n,s}^\tau \max\left\{Q_{n,s}^\tau - \gamma_{n,s}^\tau, 0\right\} \\
\leq &(Q_{n,s}^\tau)^2 + (\gamma_{n,s}^\tau)^2 - 2Q_{n,s}^\tau\gamma_{n,s}^\tau + (\delta_{n,s}^\tau)^2 + 2\delta_{n,s}^\tau Q_{n,s}^\tau \\
\leq &(Q_{n,s}^\tau)^2 + (\gamma_{n,s}^{\max})^2 + (\delta_{n,s}^{\max})^2 - 2Q_{n,s}^\tau(\gamma_{n,s}^\tau - \delta_{n,s}^\tau).
\end{aligned} \tag{23}$$

where the second inequality is always true when $\gamma_{n,s}^\tau \geq 0$ and $\delta_{n,s}^\tau \geq 0$. Given any feasible police $\mathbf{\Pi}$, applying (23) to the drift-minus-utility term for any time frame t and the time slot $\tau \in [t, t+T-1]$, we have

$$\begin{aligned}
&\Delta_T(\mathbf{\Theta}(t)) - V\mathbb{E}\left[\sum_{\tau=t}^{t+T-1} U^\tau|\mathbf{\Theta}(t)\right] \\
=&\mathbb{E}\left[L(\mathbf{\Theta}(t+T)) - L(\mathbf{\Theta}(t)) - V\sum_{\tau=t}^{t+T-1} U^{\mathbf{\Pi},\tau}|\mathbf{\Theta}(t)\right] \\
=&\mathbb{E}\left[\frac{1}{2}\sum_{\tau=t}^{t+T-1}\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}}\left((Q_{n,s}^{\tau+1})^2 - (Q_{n,s}^\tau)^2\right) - V\sum_{\tau=t}^{t+T-1} U^{\mathbf{\Pi},\tau}|\mathbf{\Theta}(t)\right] \\
\leq&\mathbb{E}\Bigg[\frac{1}{2}\sum_{\tau=t}^{t+T-1}\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}}\left((\gamma_{n,s}^{\max})^2 + (\delta_{n,s}^{\max})^2 - 2Q_{n,s}^\tau(\gamma_{n,s}^{\mathbf{\Pi},\tau} - \delta_{n,s}^{\mathbf{\Pi},\tau})\right) \\
&- V\sum_{\tau=t}^{t+T-1} U^{\mathbf{\Pi},\tau}|\mathbf{\Theta}(t)\Bigg] \\
=&\Gamma_1 T - \mathbb{E}\Bigg[\sum_{\tau=t}^{t+T-1}\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}} Q_{n,s}^\tau(\gamma_{n,s}^{\mathbf{\Pi},\tau} - \delta_{n,s}^{\mathbf{\Pi},\tau}) \\
&+ V\sum_{\tau=t}^{t+T-1} U^{\mathbf{\Pi},\tau}|\mathbf{\Theta}(t)\Bigg].
\end{aligned} \tag{24}$$

where $\Gamma_1 = \frac{1}{2}\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}}((\gamma_{n,s}^{\max})^2 + (\delta_{n,s}^{\max})^2)$.

However, for any time frame t and the time slot $\tau \in [t, t+T-1]$, we have

$$Q_{n,s}^t - (\tau - t)\gamma_{n,s}^{\max} \leq Q_{n,s}^\tau. \tag{25}$$

Plugging (25) into (24), it can be shown that

$$\begin{aligned}
&\leq \Gamma_1 T + \sum_{\tau=t}^{t+T-1}\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}}(\tau - t)\gamma_{n,s}^{\max}(\gamma_{n,s}^{\max} - 0) \\
&- \mathbb{E}\left[\sum_{\tau=t}^{t+T-1}\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}} Q_{n,s}^t(\gamma_{n,s}^{\mathbf{\Pi},\tau} - \delta_{n,s}^{\mathbf{\Pi},\tau}) + V\sum_{\tau=t}^{t+T-1} U^{\mathbf{\Pi},\tau}|\mathbf{\Theta}(t)\right] \\
&= \Gamma_2 T - \mathbb{E}\Bigg[\sum_{\tau=t}^{t+T-1}\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}} Q_{n,s}^t(\gamma_{n,s}^{\mathbf{\Pi},\tau} - \delta_{n,s}^{\mathbf{\Pi},\tau}) \\
&+ V\sum_{\tau=t}^{t+T-1} U^{\mathbf{\Pi},\tau}|\mathbf{\Theta}(t)\Bigg].
\end{aligned} \tag{26}$$

where $\Gamma_2 = \Gamma_1 + \frac{T-1}{2}\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}}(\gamma_{n,s}^{\max})^2$, and the first inequality is always true when $\mathbb{E}[\gamma_{n,s}^{\mathbf{\Pi},\tau} - \delta_{n,s}^{\mathbf{\Pi},\tau}] \geq 0$. Moreover, due to the constraint (6), the task queue backlog can not grow to infinity, thus $\mathbb{E}[\gamma_{n,s}^{\mathbf{\Pi},\tau} - \delta_{n,s}^{\mathbf{\Pi},\tau}] \geq 0$ always holds.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:05:51 UTC from IEEE Xplore. Restrictions apply.

B. Lyapunov-based Online Algorithm

Theorem IV.1 shows the deterministic upper bound of drift-minus-utility term. Based on the Lyapunov optimization theory, we devise O2TL to make decisions to minimize the upper bound of drift-minus-utility (i.e., the right hand side of the inequality of (21)) at each time frame t, which can not only find the sub-optimal maximizer of the original OSPTM problem (14), but also maintain the stability of the entire edge network and each task queue. Since Γ_2 is a constant at each time frame t, we can get the optimization problem **P2** as follows:

$$(\mathbf{P2}) \max_{\boldsymbol{x}^t,\boldsymbol{y}^\tau,\boldsymbol{m}^\tau,\boldsymbol{\kappa}^\tau} \sum_{\tau=t}^{t+T-1} \mathbb{E}\Bigg[\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}} Q_{n,ts}^t(\gamma_{n,s}^\tau-\delta_{n,s}^\tau) + VU^\tau|\boldsymbol{\Theta}(t)\Bigg]. \tag{27}$$

$$\text{s.t.} \quad (1)\text{–}(4), (8), (15), (16)$$

Note that we assume that the computation resources of BSs are limited and the task computation demands are generated all the time, thus we do not consider the trivial case where the task queue backlog is zero and the CPU is continuously idle. In other words, we assume that $Q_{n,s}^\tau \geq \frac{\kappa_{n,s}^\tau}{f_s}, \forall n \in \mathcal{N}, \forall s \in \mathcal{S}$. If $Q_{n,s}^\tau < \frac{\kappa_{n,s}^\tau}{f_s}$, we always consider computing the task computation demands at BSs.

Remark: As we can see, we loosen the upper bound of the drift-minus-utility term from (24) to (26) in Theorem IV.1. Intuitively, for any time slot $\tau \in [t, t+T-1]$, we approximate the future task queue backlog $Q_{n,s}^\tau$ as $Q_{n,s}^t$. Such an approximation can be motivated as follows: For any slot $\tau \in [t, t+T-1]$, the future task queue backlogs such as $Q_{n,s}^{\tau+1}$ depend on the current task computation demands, the decisions of O2TL and the current task queue backlogs such as $Q_{n,s}^\tau$, which are prior knowledge over time frame $[t, t+T-1]$. Therefore, it is hard to optimize the drift-minus-utility term with unknown task computation demands. Approximating the task queue backlog $Q_{n,s}^\tau$ as $Q_{n,s}^t$ can address this problem.

Substituting $\gamma_{n,s}^\tau$, $\delta_{n,s}^\tau$ and U^τ into (27) to further convert the optimization problem **P2** to **P3**, we have

$$\begin{aligned}(\mathbf{P3})\\ &\max_{\boldsymbol{x}^t,\boldsymbol{y}^\tau,\boldsymbol{m}^\tau,\boldsymbol{\kappa}^\tau} \sum_{n=1}^{N}\sum_{s=1}^{S} \mathbb{E}\left[\sum_{\tau=t}^{t+T-1} \kappa_{n,s}^\tau \frac{Q_{n,s}^t + Va_s - Vc_{f,n,s}}{f_s}|\boldsymbol{\Theta}(t)\right]\\ &- \sum_{n=1}^{N}\sum_{s=1}^{S}\sum_{i\in\mathcal{I}_n} \mathbb{E}\left[\sum_{\tau=t}^{t+T-1} y_{n,s,i}^\tau Q_{n,s}^t d_{s,i}^\tau|\boldsymbol{\Theta}(t)\right]\\ &+ \sum_{n=1}^{N}\sum_{s=1}^{S} \mathbb{E}\left[\sum_{\tau=t}^{t+T-1} m_{n,l,s}^\tau\left(Q_{n,s}^t + V\left(a_s - c_{n,l} - c_{f,l,s}\right)\right)|\boldsymbol{\Theta}(t)\right]\\ &- \sum_{n=1}^{N}\sum_{s=1}^{S} \mathbb{E}\left[\sum_{\tau=t}^{t+T-1} x_{n,s}^t \frac{Vc_{s,n}(1-x_{n,s}^{t-T})}{T}|\boldsymbol{\Theta}(t)\right].\end{aligned}$$

$$\text{s.t.} \quad (1)-(4), (8), (15), (16). \tag{28}$$

As we can see, the problem **P3** is a T-time-slot optimization problem over any time frame $t = kT$, where $1 - x_{n,s}^{t-T} = 1$ when $t = 0$. On the other hand, the task queue backlogs $\boldsymbol{\Theta}(t)$ are unchanged over the time slot $\tau \in [t, t+T-1]$. Therefore, we can decompose the T-time-slot optimization problem into multiple one-time-slot *mixed integer linear programming* optimization problems. We can find the solution effectively by solvers (e.g., Gurobi [39]) as in [40], [41], [42]. For each one-time-slot optimization problem, only the variable $d_{s,i}^\tau$ affects finding the optimal solution.

Then, we design the O2TL algorithm to solve the OSPTM problem with the decision variables $\boldsymbol{x}^t, \boldsymbol{y}^\tau, \boldsymbol{m}^\tau, \boldsymbol{\kappa}^\tau$ at each time slot $\tau \in [t, t+T-1]$, where $d_{s,i}^\tau$ is a constant at current time slot. Note that the task computation demand $d_{s,i}^\tau$ is a known constant at the beginning of each time slot, while all $d_{s,i}^\tau$ over the time slot $\tau \in [t, t+T-1]$ are unknown at the beginning of time frame t. In addition, $\boldsymbol{x}^t$ is only determined at time frame t and $x_{n,s}^{t-T}$ is a known constant. The proposed O2TL algorithm is sketched in Algorithm 1.

C. Performance Analysis

First, we consider the stationary and randomized policies, which only observe task computation demand $d_{s,i}^\tau$ for each slot τ and independently make feasible decisions $\boldsymbol{x}^t, \boldsymbol{y}^t, \boldsymbol{m}^t, \boldsymbol{\kappa}^t$.

Lemma IV.1. (Optimality over Stationary and Randomized Policies). Suppose the task computation demand $d_{s,i}^\tau$ follows an exponential distribution with the parameter $1/\lambda_{s,i}$. For any $\phi > 0$, there exists an independent, stationary and randomized policy $\boldsymbol{\pi}$ that makes decisions $\boldsymbol{x}^t, \boldsymbol{y}^\tau, \boldsymbol{m}^\tau, \boldsymbol{\kappa}^\tau$ for any $t = kT$ and $\tau \in [t, t+T-1]$, and

$$\begin{aligned}&\mathbb{E}\left[\sum_{\tau=t}^{t+T-1}\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}} \gamma_{n,s}^{\boldsymbol{\pi},\tau}\right] \geq \mathbb{E}\left[\sum_{\tau=t}^{t+T-1}\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}} \delta_{n,s}^{\boldsymbol{\pi},\tau}\right] + \phi,\\ &\mathbb{E}\left[\sum_{\tau=t}^{t+T-1} U^{\boldsymbol{\pi},\tau}\right] \geq TU^{opt} + \phi.\end{aligned} \tag{29}$$

where U^{opt} is the expectation of the supremum time-average network utility over all feasible policies.

Proof. It can be proven by using Theorem 4.5 in [22], which is omitted here for brevity.

Then, we analyze the performance of O2TL based on Lemma IV.1.

Theorem IV.2. (Performance of O2TL). Suppose the task computation demand $d_{s,i}^\tau$ follows the exponential distribution with the parameter $1/\lambda_{s,i}$, and $\mathbb{E}[L(\boldsymbol{\Theta}(0))] < \infty$. Given any fixed control parameter $V > 0$, $\phi > 0$, we have

$$\limsup_{|\mathcal{T}|\to\infty} \frac{1}{|\mathcal{T}|}\mathbb{E}\left[\sum_{\tau=0}^{|\mathcal{T}|-1} U^{O2TL,\tau}\right] \geq U^{opt} - \frac{\Gamma_2}{V}, \tag{30}$$

$$\limsup_{K\to\infty} \frac{1}{K}\mathbb{E}\left[\sum_{t=0}^{KT}\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}} Q_{n,s}^t\right] \leq \frac{\Gamma_2 V\left(U^{\max} - U^{opt}\right)}{\phi}, \tag{31}$$

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:05:51 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: O2TL.

Input: Task computation demand $d_{s,i}^{\tau}$,
Revenue a_s, Service placement cost $c_{s,n}$,
Task migration cost $c_{n,l}$,
Task computation cost $c_{f,n,s}$ and $c_{f,l,s}$
Output: A feasible police with decisions $\boldsymbol{x}^t, \boldsymbol{y}^{\tau}, \boldsymbol{m}^{\tau}, \boldsymbol{\kappa}^{\tau}$
1: Set the time iteration counter $\tau = 0$ and $t = 0$
2: **repeat**
3: **if** $\tau = kT$ **then**
4: Set time frame $t = \tau$
5: Observe the queue backlog $\boldsymbol{\Theta}(t)$, the service placement decision $\boldsymbol{x}^{t-T}$ (if $t = 0$, $\boldsymbol{x}^{t-T} = 0$) and the task computation demand $d_{s,i}^{t}$
6: Make decisions $\boldsymbol{x}^t, \boldsymbol{y}^t, \boldsymbol{m}^t, \boldsymbol{\kappa}^t$ by solving problem **P4**:

$$\max_{\boldsymbol{x}^t,\boldsymbol{y}^{\tau},\boldsymbol{m}^{\tau},\boldsymbol{\kappa}^{\tau}} \sum_{n=1}^{N}\sum_{s=1}^{S}\mathbb{E}\left[\kappa_{n,s}^{\tau}\frac{Q_{n,s}^{t}+Va_s-Vc_{f,n,s}}{f_s}|\boldsymbol{\Theta}(t)\right]$$
$$-\sum_{n=1}^{N}\sum_{s=1}^{S}\sum_{i\in\mathcal{I}_n}\mathbb{E}\left[y_{n,s,i}^{\tau}Q_{n,s}^{t}d_{s,i}^{\tau}|\boldsymbol{\Theta}(t)\right]$$
$$+\sum_{n=1}^{N}\sum_{s=1}^{S}\mathbb{E}\left[m_{n,l,s}^{\tau}\left(Q_{n,s}^{t}+V\left(a_s-c_{n,l}-c_{f,l,s}\right)\right)|\boldsymbol{\Theta}(t)\right]$$
$$-\sum_{n=1}^{N}\sum_{s=1}^{S}\mathbb{E}\left[x_{n,s}^{t}\frac{Vc_{s,n}(1-x_{n,s}^{t-T})}{T}|\boldsymbol{\Theta}(t)\right]$$
$$\text{s.t. } (1)-(4),(8),(15),(16)$$

7: Update the queue $\boldsymbol{\Theta}(t+1)$
8: **else**
9: Get time slot τ
10: Observe the queue backlog $\boldsymbol{\Theta}(t)$, the service placement decision $\boldsymbol{x}^t$ and the task computation demand $d_{s,i}^{\tau}$
11: Make decisions $\boldsymbol{y}^{\tau}, \boldsymbol{m}^{\tau}, \boldsymbol{\kappa}^{\tau}$ by solving problem **P5**:

$$\max_{\boldsymbol{y}^{\tau},\boldsymbol{m}^{\tau},\boldsymbol{\kappa}^{\tau}} \sum_{n=1}^{N}\sum_{s=1}^{S}\mathbb{E}\left[\kappa_{n,s}^{\tau}\frac{Q_{n,s}^{t}+Va_s-Vc_{f,n,s}}{f_s}|\boldsymbol{\Theta}(t)\right]$$
$$-\sum_{n=1}^{N}\sum_{s=1}^{S}\sum_{i\in\mathcal{I}_n}\mathbb{E}\left[y_{n,s,i}^{\tau}Q_{n,s}^{t}d_{s,i}^{\tau}|\boldsymbol{\Theta}(t)\right]$$
$$+\sum_{n=1}^{N}\sum_{s=1}^{S}\mathbb{E}\left[m_{n,l,s}^{\tau}\left(Q_{n,s}^{t}+V\left(a_s-c_{n,l}-c_{f,l,s}\right)\right)|\boldsymbol{\Theta}(t)\right]$$
$$\text{s.t. } (2)-(4),(8),(16)$$

12: Update the queue $\boldsymbol{\Theta}(\tau+1)$
13: **end if**
14: $\tau \leftarrow \tau + 1$
15: **until** $\tau >$ *Maximum value set in advance*

where Γ_2 is shown in Theorem IV.1, and $U^{\max}$ is defined as the maximum network utility, which is different from the expectation of the supremum time-average network utility U^{opt}. However, it holds that $U^{opt} \leq U^{\max}$.

Proof. Recall that O2TL seeks to make decisions to minimize the right hand side of the inequality of (21). For any time frame $t = kT (k \in \mathbb{N})$ and the time slot $\tau \in [t, t+T-1]$, according to Theorem IV.1, we have

$$\Delta_T(\boldsymbol{\Theta}(t)) - V\mathbb{E}\left[\sum_{\tau=t}^{t+T-1} U^{O2TL,\tau}|\boldsymbol{\Theta}(t)\right]$$
$$\leq \Gamma_2 T - \mathbb{E}\left[\sum_{\tau=t}^{t+T-1}\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}} Q_{n,s}^{t}(\gamma_{n,s}^{\boldsymbol{\Pi},\tau}-\delta_{n,s}^{\boldsymbol{\Pi},\tau}) + V\sum_{\tau=t}^{t+T-1} U^{\boldsymbol{\Pi},\tau}|\boldsymbol{\Theta}(t)\right], \tag{32}$$

where $\boldsymbol{\Pi}$ represents any alternative feasible policy that can be implemented over the time frame t. Now fix $\phi > 0$, and consider an independent, stationary and randomized policy $\boldsymbol{\pi}$ based on Lemma IV.1. Because the task computation demand $d_{s,i}^{\tau}$ follows an exponential distribution over time slots, the resulting values of $\gamma_{n,s}^{\boldsymbol{\pi},\tau}$, $\delta_{n,s}^{\boldsymbol{\pi},\tau}$, $U^{\boldsymbol{\pi},\tau}$ are independent of the current queue backlogs $\boldsymbol{\Theta}(t)$, and we have

$$\mathbb{E}\left[\sum_{\tau=t}^{t+T-1}\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}}\gamma_{n,s}^{\boldsymbol{\pi},\tau}|\boldsymbol{\Theta}(t)\right] = \mathbb{E}\left[\sum_{\tau=t}^{t+T-1}\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}}\gamma_{n,s}^{\boldsymbol{\pi},\tau}\right]$$
$$\geq \mathbb{E}\left[\sum_{\tau=t}^{t+T-1}\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}}\delta_{n,s}^{\boldsymbol{\pi},\tau}|\boldsymbol{\Theta}(t)\right] + \phi$$
$$= \mathbb{E}\left[\sum_{\tau=t}^{t+T-1}\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}}\delta_{n,s}^{\boldsymbol{\pi},\tau}\right] + \phi,$$
$$\mathbb{E}\left[\sum_{\tau=t}^{t+T-1} U^{\boldsymbol{\pi},\tau}|\boldsymbol{\Theta}(t)\right] = \mathbb{E}\left[\sum_{\tau=t}^{t+T-1} U^{\boldsymbol{\pi},\tau}\right] \geq TU^{opt} + \phi. \tag{33}$$

Using Lemma IV.1 and substituting the inequalities (33) into the inequality (32), we have

$$\Delta_T(\boldsymbol{\Theta}(t)) - V\mathbb{E}\left[\sum_{\tau=t}^{t+T-1} U^{O2TL,\tau}|\boldsymbol{\Theta}(t)\right]$$
$$\leq \Gamma_2 T - \mathbb{E}\left[\sum_{\tau=t}^{t+T-1}\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}} Q_{n,s}^{t}(\gamma_{n,s}^{\boldsymbol{\pi},\tau}-\delta_{n,s}^{\boldsymbol{\pi},\tau}) + V\sum_{\tau=t}^{t+T-1} U^{\boldsymbol{\pi},\tau}|\boldsymbol{\Theta}(t)\right]$$
$$\leq \Gamma_2 T - T\phi\mathbb{E}\left[\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}} Q_{n,s}^{t}|\boldsymbol{\Theta}(t)\right] - V\left(TU^{opt}+\phi\right). \tag{34}$$

Further, from the above drift inequality (34), rearranging the terms, summing the inequality over $\tau \in \mathcal{T} = \{0, 1, \ldots, KT-1\}$ and letting $\phi \to 0$, we have

$$V\mathbb{E}\left[\sum_{\tau=0}^{KT-1} U^{O2TL,\tau}\right]$$

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:05:51 UTC from IEEE Xplore. Restrictions apply.

$$\geq VKTU^{opt} - K\Gamma_2 T + L(\mathbf{\Theta}(KT)) - L(\mathbf{\Theta}(0))$$
$$\geq VKTU^{opt} - K\Gamma_2 T - L(\mathbf{\Theta}(0)). \quad (35)$$

Dividing both sides of (35) by KTV , we have

$$\frac{1}{KT}\mathbb{E}\left[\sum_{\tau=0}^{KT-1} U^{O2TL,\tau}\right] \geq U^{opt} - \frac{\Gamma_2}{V} - \frac{L(\mathbf{\Theta}(0))}{KTV}, \quad (36)$$

which proves (30) by taking a $\limsup$ as $KT \to \infty$ ($KT = |\mathcal{T}|$).

Similarly, from the above drift inequality (34), rearranging the terms, summing the inequality over $t = KT$, we have

$$T\phi\mathbb{E}\left[\sum_{t=0}^{KT}\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}} Q_{n,s}^t\right] \leq K\Gamma_2 T + V\mathbb{E}\left[\sum_{\tau=0}^{KT-1} U^{O2TL,\tau}\right] - VK\left(TU^{opt}+\phi\right) - L(\mathbf{\Theta}(KT)) + L(\mathbf{\Theta}(0))$$
$$= K\Gamma_2 T + V\mathbb{E}\left[\sum_{\tau=0}^{KT-1} U^{O2TL,\tau}\right] - V\mathbb{E}\left[\sum_{\tau=0}^{KT-1} U^{opt}\right] + L(\mathbf{\Theta}(0))$$
$$\leq K\Gamma_2 T + V\mathbb{E}\left[\sum_{\tau=0}^{KT-1}\left(U^{O2TL,\tau} - U^{opt}\right)\right] + L(\mathbf{\Theta}(0)). \quad (37)$$

Dividing both sides of (37) by $KT\phi$, we have

$$\frac{1}{K}\mathbb{E}\left[\sum_{t=0}^{KT}\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}} Q_{n,s}^t\right]$$
$$\leq \frac{\Gamma_2}{\phi} + \frac{V}{KT\phi}\mathbb{E}\left[\sum_{\tau=0}^{KT-1}\left(U^{O2TL,\tau} - U^{opt}\right)\right] + \frac{L(\mathbf{\Theta}(0))}{KT\phi}$$
$$\leq \frac{\Gamma_2}{\phi} + \frac{V\left(U^{\max} - U^{opt}\right)}{\phi} + \frac{L(\mathbf{\Theta}(0))}{KT\phi}. \quad (38)$$

which proves (31) by taking a $\limsup$ as $K \to \infty$.

Theorem IV-C presents a performance-backlog tradeoff of $[O(1/V), O(V)]$, where $V > 0$ is a control parameter. The inequality (30) shows that the lower bound of the expectation of the time-average network utility increases as V increases. However, the inequality (31) shows that the upper bound of the task queue backlogs increases as V increases. By setting a large value of V, O2TL can reach high network utility, while the task queue backlogs would also be large. Besides, we can set ε as $\frac{\Gamma_2 V(U^{\max}-U^{opt})}{\phi}$ in (6).

D. Extension to Variable $V(t)$

From the $[O(1/V), O(V)]$ performance-backlog tradeoff, we can find that if we use a control *variable* parameter $V(t)$ that gradually increases with time instead of the fixed parameter V, we can maintain the mean rate stability of task queue backlogs while driving the time average expectation network utility to its optimum value U^{opt}.

Note that the number of feasible policies to minimize the upper bound of the drift-minus-utility term is large (even approaching infinite), so that the upper bound of the drift-minus-utility term may not have a well defined maximum. Nevertheless, we introduce Ω-additive approximation as follows to analyze the upper bound of the drift-minus-utility term within an additive constant Ω of the infimum. In other words, we allow an error Ω to enhance the robustness of the O2TL algorithm.

Definition 11. (Ω-additive Approximation). At each time frame t and over the time slot $\tau \in [t, t+T-1]$, for a given constant $\Omega \geq 0$ and the current $\mathbf{\Theta}(t)$, an Ω-additive approximation of the O2TL algorithm chooses a randomized feasible policy $\mathbf{\Pi}$ that yields a conditional expected value of the right hand side of the drift-minus-utility term that is within a constant Ω from the infimum over all possible feasible policies.

Then, we analyze the performance of the O2TL algorithm based on Definition 11 and control variable parameter $V(t)$.

Theorem IV.3. (Performance of O2TL with Extension to Variable $V(t)$). Suppose the task computation demand $d_{s,i}^{\tau}$ follows the exponential distribution with the parameter $1/\lambda_{s,i}$, and $\mathbb{E}[L(\mathbf{\Theta}(0))] < \infty$. Suppose at every time slot $\tau \in [t, t+T-1]$, we implement an Ω-additive approximation that comes within $\Omega \geq 0$ of the infimum of a modified right hand side of (32), where the parameter V is replaced with $V(t)$ defined as

$$V(t) \triangleq V_0(t+T)^{\beta}, \forall t = kT, \quad (39)$$

for some constants $V_0 > 0$ and $\beta (0 < \beta < 1)$. Then all task queue backlogs are mean rate stable, all required constraints (1), (2), (3), (4),(8),(15),(16) are satisfied, and

$$\limsup_{|\mathcal{T}|\to\infty} \frac{1}{|\mathcal{T}|}\mathbb{E}\left[\sum_{\tau=0}^{|\mathcal{T}|-1} U^{O2TL,\tau}\right] = U^{opt}. \quad (40)$$

Note that how V_0 and β affect the convergence is described in the proof.

Proof. Similar to the proof of Theorem IV.2, for any time frame $t = kT (k \in \mathbb{N})$ and the time slot $\tau \in [t, t+T-1]$, according to Theorem IV.1 and Lemma IV.1, we have

$$\Delta_T(\mathbf{\Theta}(t)) - V(t)\mathbb{E}\left[\sum_{\tau=t}^{t+T-1} U^{O2TL,\tau}|\mathbf{\Theta}(t)\right]$$
$$\leq \Gamma_2 T + \Omega T - \mathbb{E}\left[\sum_{\tau=t}^{t+T-1}\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}} Q_{n,s}^t(\gamma_{n,s}^{\boldsymbol{\pi},\tau} - \delta_{n,s}^{\boldsymbol{\pi},\tau}) + V(t)\sum_{\tau=t}^{t+T-1} U^{\boldsymbol{\pi},\tau}|\mathbf{\Theta}(t)\right]$$
$$\leq \Gamma_2 T + \Omega T - T\phi\mathbb{E}\left[\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}} Q_{n,s}^t|\mathbf{\Theta}(t)\right] - V(t)\left(TU^{opt}+\phi\right). \quad (41)$$

Taking $\phi \to 0$, we have

$$\Delta_T(\mathbf{\Theta}(t)) - V(t)\mathbb{E}\left[\sum_{\tau=t}^{t+T-1} U^{O2TL,\tau}|\mathbf{\Theta}(t)\right]$$
$$\leq \Gamma_2 T + \Omega T - V(t)TU^{opt}. \quad (42)$$

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:05:51 UTC from IEEE Xplore. Restrictions apply.

Taking expectations on both sides and rearranging the terms, we have

$$\begin{aligned}&\mathbb{E}\left[L(\mathbf{\Theta}(t+T))\right]-\mathbb{E}\left[L(\mathbf{\Theta}(t))\right]\leq\Gamma_2T+\Omega T\\&-V(t)TU^{opt}+V(t)\mathbb{E}\left[\sum_{\tau=t}^{t+T-1}U^{O2TL,\tau}\right]\\&\leq\Gamma_2T+\Omega T+V(t)T\left(U^{\max}-U^{opt}\right).\end{aligned}\tag{43}$$

Summing over $t=kT(k\in\mathbb{N})$, we have

$$\begin{aligned}&\mathbb{E}\left[L(\mathbf{\Theta}(kT))\right]-\mathbb{E}\left[L(\mathbf{\Theta}(0))\right]\\&\leq k\Gamma_2T+k\Omega T+T\left(U^{\max}-U^{opt}\right)\sum_{t=0}^{kT-T}V(t).\end{aligned}\tag{44}$$

Using the definition of the Lyapunov function in (18), we have

$$\begin{aligned}&\frac{1}{2}\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}}\mathbb{E}\left[(Q_{n,s}^{kT})^2\right]\\&\leq k\Gamma_2T+k\Omega T+\mathbb{E}\left[L(\mathbf{\Theta}(0))\right]+T\left(U^{\max}-U^{opt}\right)\sum_{t=0}^{kT-T}V(t).\end{aligned}\tag{45}$$

For any queue $Q_{n,s}^{kT}$, because $\mathbb{E}[(Q_{n,s}^{kT})^2]\leq\mathbb{E}[(Q_{n,s}^{kT})]^2$, we have

$$\begin{aligned}&\mathbb{E}\left[(Q_{n,s}^{kT})\right]\leq\\&\sqrt{2kT(\Gamma_2+\Omega)+2\mathbb{E}[L(\mathbf{\Theta}(0))]+2T(U^{\max}-U^{opt})\sum_{t=0}^{kT-T}V(t)}.\end{aligned}\tag{46}$$

Dividing both sides of the above inequality by kT, we have

$$\begin{aligned}&\frac{\mathbb{E}\left[(Q_{n,s}^{kT})\right]}{kT}\leq\\&\sqrt{\frac{2(\Gamma_2+\Omega)}{kT}+\frac{2\mathbb{E}\left[L(\mathbf{\Theta}(0))\right]}{(kT)^2}+\frac{2T(U^{\max}-U^{opt})}{(kT)^2}\sum_{t=0}^{kT-T}V(t)}.\end{aligned}\tag{47}$$

In addition, we have

$$\begin{aligned}0&\leq\frac{1}{(kT)^2}\sum_{t=0}^{kT-T}V(t)=\frac{V_0}{(kT)^2}\sum_{t=0}^{kT-T}(t+T)^\beta\\&=\frac{V_0}{(kT)^2}\sum_{k=0}^{k-1}(kT+T)^\beta\leq\frac{V_0}{(kT)^2}\int_0^k(T+Tv)^\beta dv\\&=\frac{V_0}{(kT)^2}\left[\frac{(T+Tk)^{\beta+1}-(T)^{\beta+1}}{(\beta+1)T}\right].\end{aligned}\tag{48}$$

Because $0<\beta<1$, when taking a limit of (48) as $k\to\infty$, the growth rate of the numerator is slower than that of the denominator, so it shows that $\frac{\sum_{t=0}^{kT-T}V(t)}{(kT)^2}\to 0$. Thus, taking a limit of (47) shows that all queues are mean rate stable.

To prove that the expectation of the time-average network utility converges to U^{opt}, taking expectations on both sides of (42) and dividing both sides of (42) by $V(t)$, we have

$$\begin{aligned}&\frac{\mathbb{E}\left[L(\mathbf{\Theta}(t+T))\right]-\mathbb{E}\left[L(\mathbf{\Theta}(t))\right]}{V(t)}-\mathbb{E}\left[\sum_{\tau=t}^{t+T-1}U^{O2TL,\tau}\right]\\&\leq\frac{(\Gamma_2+\Omega)T}{V(t)}-TU^{opt}.\end{aligned}\tag{49}$$

Summing over $t=kT(k\in\mathbb{N})$, we have

$$\begin{aligned}&\frac{\mathbb{E}\left[L(\mathbf{\Theta}(kT))\right]}{V(kT-T)}-\frac{\mathbb{E}\left[L(\mathbf{\Theta}(0))\right]}{V(0)}-\mathbb{E}\left[\sum_{\tau=0}^{kT-1}U^{O2TL,\tau}\right]\\&+\sum_{t=T}^{kT-T}\mathbb{E}\left[L(\mathbf{\Theta}(t))\right]\left[\frac{1}{V(t-T)}-\frac{1}{V(t)}\right]\\&\leq(\Gamma_2+\Omega)T\sum_{t=0}^{kT-T}\frac{1}{V(t)}-kTU^{opt}.\end{aligned}\tag{50}$$

Because $V(t)$ is non-decreasing, we have for all $t\geq T$

$$\left[\frac{1}{V(t-T)}-\frac{1}{V(t)}\right]\geq 0.\tag{51}$$

According to the above inequality, rearranging and dividing both sides of (50) by kT, we have

$$\begin{aligned}&\frac{1}{kT}\mathbb{E}\left[\sum_{\tau=0}^{kT-1}U^{O2TL,\tau}\right]\\&\geq U^{opt}-\frac{(\Gamma_2+\Omega)T}{kT}\sum_{t=0}^{kT-T}\frac{1}{V(t)}-\frac{\mathbb{E}\left[L(\mathbf{\Theta}(0))\right]}{kTV(0)}.\end{aligned}\tag{52}$$

Moreover, we have

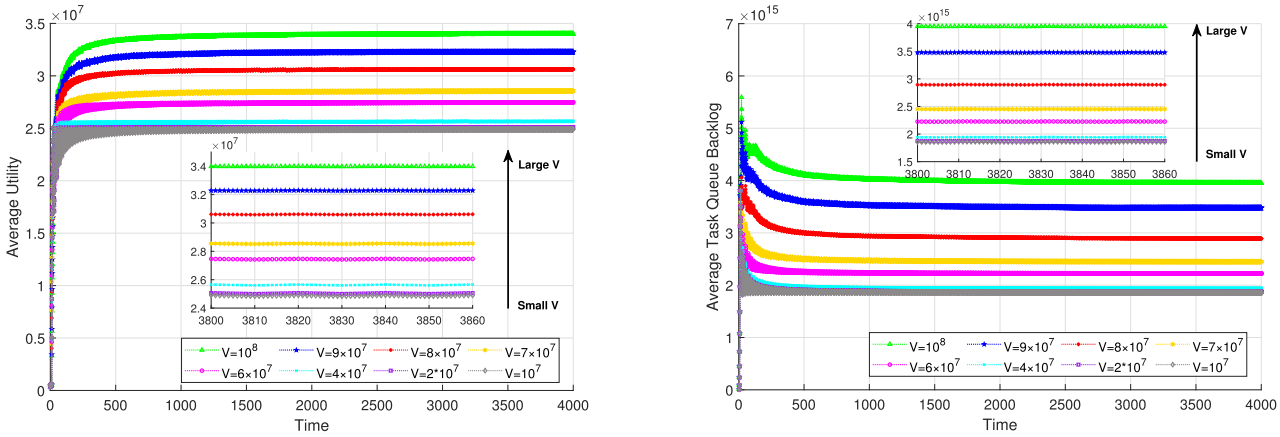
$$\begin{aligned}0&\leq\frac{1}{kT}\sum_{t=0}^{kT-T}\frac{1}{V(t)}\\&\leq\frac{1}{kT}\frac{1}{V(0)}+\frac{1}{V_0kT}\int_0^k\frac{1}{(T+Tv)^\beta}dv\\&=\frac{1}{kT}\frac{1}{V(0)}+\frac{1}{V_0kT}\left[\frac{(T+Tk)^{1-\beta}-(T)^{1-\beta}}{(1-\beta)T}\right].\end{aligned}\tag{53}$$

Taking a limit as $k\to\infty$ shows that the term $\frac{1}{kT}\sum_{t=0}^{kT-T}\frac{1}{V(t)}$ vanishes, and the left hand side of (52) is more than or equal to U^{opt}. On the other hand, U^{opt} is the supremum time average expectation network utility over all feasible policies, that is, the left hand side of (52) is less than or equal to U^{opt}. Therefore, the limit exists and is equal to U^{opt}, which proves (40).

V. Performance Evaluation

In this section, we conduct extensive real-world data-driven simulations to evaluate the performance of the proposed O2TL algorithm. All the algorithms are implemented on an Intel Core i5-8265U CPU @1.60GHz with 8GB RAM.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:05:51 UTC from IEEE Xplore. Restrictions apply.



(a) Time v.s. Average Utility

(b) Time v.s. Average Task Queue Backlog

Fig. 3. Average Utility and Task Queue Backlog with Different Trade-off Parameter V ($T = 10$).

TABLE III
SYSTEM PARAMETERS

Parameter	Value
Number of base stations	9
Number of disjoint individual subareas	25
Number of computing services	10
Maximum CPU frequency of BS, F_n	[100,150] GHz
Storage capacity of BS, R_n	[200,280] GB
Storage space required by service, r_s	[30,80] GB
CPU cycles required per bit, f_s	[0.0001,0.0005] GHz/bit
Revenue obtained per bit, a_s	[0.07,0.1] $/bit
Service placement cost, $c_{s,n}$	[5,8] $
Task computation cost on BS per bit, $c_{f,n,s}$	[0.02,0.04] $/bit
Task computation cost on cloud per bit, $c_{f,l,s}$	[0.01,0.06] $/bit
BS-Cloud task migration cost per bit, $c_{n,l}$	[0.02,0.05] $/bit
Task computation demands, $\lambda_{s,i}$	5×10^5

A. Simulation Setup

1) Dataset: Since the information of task demands in public BSs is often confidential, we generate the simulation dataset from Google cluster-usage traces [43], which records three million task arrivals at one of its server clusters of machines within one month in May 2011. Each task arrival contains the arrival time, task ID, job ID and the CPU consumption, while each machine has the machine ID, CPU capacity and memory capacity. In the experiment, we regard each job as a specific service task demand and each machine as a BS. Note that, due to the privacy protection, the value of CPU consumption, CPU capacity and memory capacity are not actual value but a linear transformation of the true values.

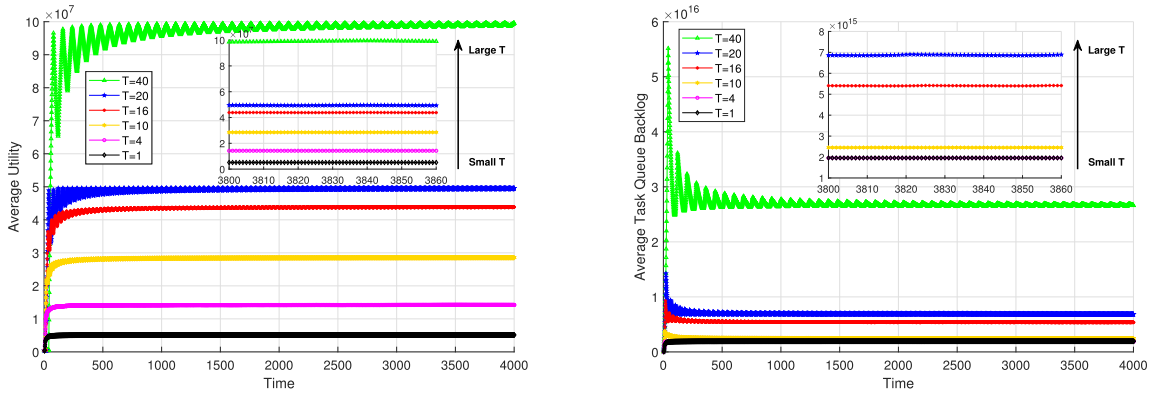
2) MEC System Detail: Specifically, we consider an edge network with $N = 9$ BSs uniformly deployed in a $500m \times 500m$ area, which is divided into 25 disjoint individual subareas with 10 computing services. The task computation demand follows an exponential distribution with the parameter $\lambda_{s,i} = 5 \times 10^5$. Other main parameters are shown in Table III.

B. Performance of O2TL

1) The Impact of Tradeoff Parameter V: We first evaluate the impact of different tradeoff parameter V on the average utility and the average task queue backlog over time, as shown in Fig. 3. We set time frame length $T = 10$ over 4000 time slots and the value of V ranging from 10^7 to 10^8. As can be seen from Fig. 3(a) and (b), the values of the average utility and the average task queue backlog gradually become stable over time for each V. Furthermore, it can be shown from the partially enlarged views that the value of the average utility (or the task queue backlog) fluctuates slightly in a fixed value range with the time frame length T. It is because that at the beginning of the time frame t, some task queue backlogs of task demands for non-deployed services will be larger than others, that is the task queue backlog $Q^t_{n,s}$ will affect more in the problem **P2**. Moreover, during the time slot $\tau \in [t, t+T-1]$, the MEC system focuses more on minimizing the task queue backlog, so the value of the average utility and the average task queue backlog will decrease slightly. On the other hand, the MEC system will tend to deploy services with larger task queue backlogs at time frame t, so that the task queue backlogs for these services will be reduced during time slot $\tau \in [t, t+T-1]$. At the beginning of next time frame $t+T$, the value of task queue backlogs for deployed services will decrease, then the tradeoff parameter V will affect more in the problem **P2**. In other words, the MEC system now pays more attention to maximizing the network utility, so the value of average utility and average task queue backlog will increase slightly. This process repeats, leading to the fluctuation in Fig. 3.

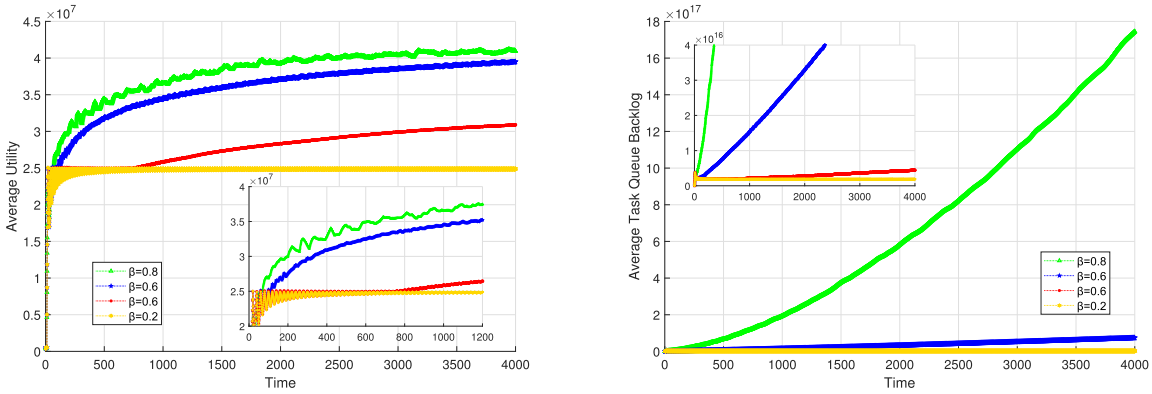
Furthermore, the average utility and the average task queue backlog increase as V increases, which is consistent with Theorem IV.2. In other words, if the MEC system wants to obtain higher network utility, the task queue backlog will also becomes larger, that is, the average delay of the task demands will increase. According to Little's Law [44], the average delay experienced by each task demand is proportional to the average task queue backlog in the MEC system, which can be calculated by $\sum_{n\in\mathcal{N}}\sum_{s\in\mathcal{S}} Q^t_{n,s} / \sum_{s\in\mathcal{S}}\sum_{i\in\mathcal{I}} \lambda_{s,i}$. Notably, when the value

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:05:51 UTC from IEEE Xplore. Restrictions apply.



(a) Time v.s. Average Utility (b) Time v.s. Average Task Queue Backlog

Fig. 4. Average Utility and Task Queue Backlog with Different Time Frame Length T ($V = 7 \times 10^7$).



(a) Time v.s. Average Utility (b) Time v.s. Average Task Queue Backlog

Fig. 5. Average Utility and Task Queue Backlog with Trade-off Parameter $V(t)$ ($T = 10, V_0 = 5 \times 10^6$).

of V is less than 10^7, the result does not change significantly. This is because when the value of V is comparable to the magnitude difference between the task queue backlog and the cost (revenue), it will have a significant impact on the result.

2) The Impact of Time Frame Length T: We evaluate the impact of different time frame length T on the average utility and the average task queue backlog, as shown in Fig. 4. We set tradeoff parameter $V = 7 \times 10^7$ and the value of T ranging from 1 to 40 over 4000 time slots, which is a sufficiently large range for exploring the characteristics of O2TL. Intuitively, the larger T corresponds to the larger task queue backlogs for none-deployed services. That is because during the time frame with length T, the certain service placement decisions are hard to accommodate the the task demands that dynamically change at each time slot. Therefore, choosing an appropriate T can obtain more network utility through a small increase in task queue backlog. As can be seen from Fig. 4(a) and (b), the values of the average utility and the average task queue backlog also gradually become stable over time for each T and fluctuate slightly in a fixed value range with the time frame length T.

Specifically, when T increases, the network utility also increases while causing a large task queue backlog. The value of T can be set according to the maximum tolerant delay of the edge network. For example, we can see from Fig. 4 that, when we set $T = 10$, a sacrifice of only around 25% yields a 680% increase in network utility compared with $T = 1$. Moreover, when we set $T = 40$, although the network utility increases by 1900%, but it brings about task queue backlog growth of 1250%. Obviously, in our experiment, $T = 10$ is the better choice.

3) The Impact of Trade-off $V(t)$: Here, we investigate the impact of the variable tradeoff parameter $V(t) \triangleq V_0(t+T)^\beta$ on the performance of O2TL. We set $V_0 = 5 \times 10^6$ and $\beta = \{0.2, 0.4, 0.6, 0.8\}$ over 4000 time slots. As shown in Fig. 5, the values of the average utility and the average task queue backlog show an upward trend where the growth rate gradually decreases over time, which is consistent with (52) in Theorem IV.3. Furthermore, we can see that when $\beta = 0.2$, due to the small growth rate of $V(t)$, the value of V is around 2×10^7 over 4000 time slots, thereby the values of the average utility and the average task queue backlog are stable (similar to the previous results). On the contrary, when $\beta = 0.8$, V grows quickly over 4000 time slots, resulting in a larger growth rate of task queue backlog. However, as time goes by, the growth rate of V slows down, and the growth rate of task queue backlog will also slow down. The final growth trend will be mean rate stable.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:05:51 UTC from IEEE Xplore. Restrictions apply.

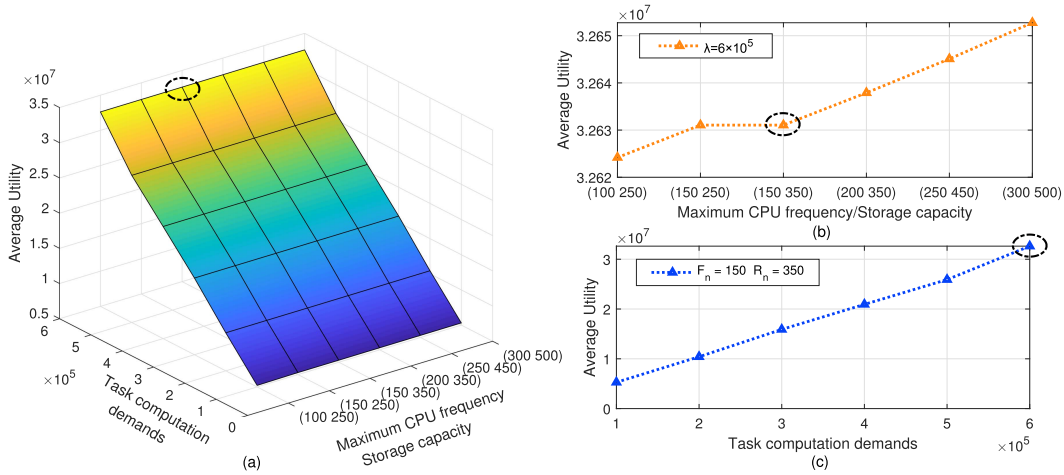


Fig. 6. Average Utility versus Maximum CPU Frequency F_n/Storage Capacity R_n and Task computation demands $\lambda_{s,i}$ ($t = 4000$).

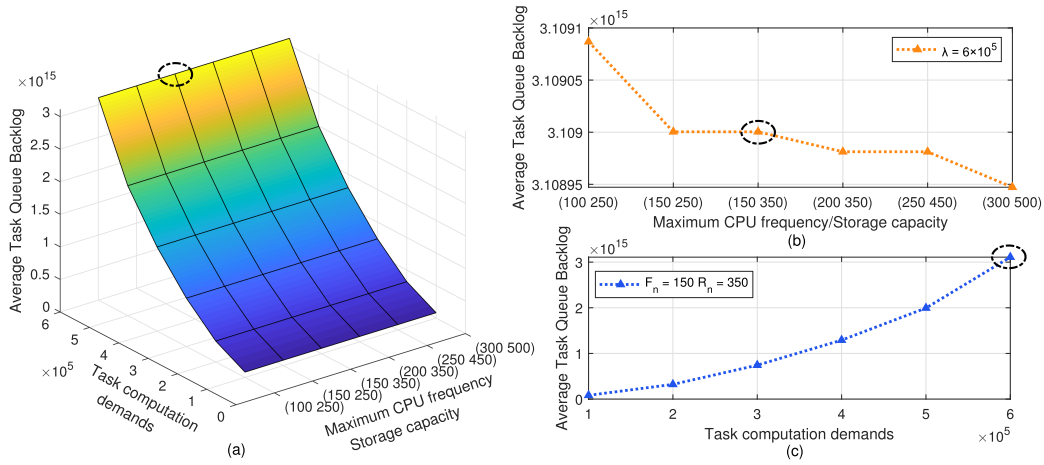


Fig. 7. Average Task Queue Backlog versus Maximum CPU Frequency F_n/Storage Capacity R_n and Task computation demands $\lambda_{s,i}$ ($t = 4000$).

4) The Impact of Maximum CPU Frequency F_n, Storage Capacity R_n and Task Computation Demands $\lambda_{s,i}$: The above experiments have shown the impacts of tradeoff parameter V and time frame length T of the O2TL algorithm on the edge network. In order to further verify the performance of O2TL in different network environments, we will examine the network utility and the task queue backlog more deeply by controlling other parameters such as maximum CPU frequency, storage capacity and task computation demands. We set $\lambda_{s,i}$ from 10^5 to 6×10^5, F_n from 100 to 300 and R_n from 250 to 500. For the maximum CPU frequency and storage capacity, we select six groups of parameters that can examine the performance of O2TL. In addition, we assume that the BSs are homogeneous with the same maximum CPU frequency and storage capacity. As shown in Fig. 6(a), the average utility increases as the task computation demands increase, which is obvious. For instance, the result is shown in Fig. 6(c) with the parameters $F_n = 150$ and $R_n = 350$. On the other hand, as the maximum CPU frequency and storage capacity increase, the average utility also shows an upward trend. Specifically, as shown in Fig. 6(b), when the maximum CPU frequency is constant and the storage capacity increases, the network utility may not increase, such as $(F_n, R_n) = (150, 250)$ and $(F_n, R_n) = (150, 350)$. This is because even if there are many services that can be deployed, the maximum CPU frequency is still limited, so that only limited task demands can be processed at BSs. On the contrary, when the storage capacity is constant and the maximum CPU frequency increases, more task demands can be processed at the edge network, so the utility will increase.

As shown in Fig. 7(a), the average task queue backlog shows the similar upward trend compared with the aforementioned results when the task computation demands increase. It can be seen from Fig. 7(c) that when the task increases from 10^5 to 6×10^5, the average task queue backlog also increases linearly. This is consistent with (31) in Theorem IV.2, where Γ_2 determines the growth rate of task queue backlog when V is certain. Furthermore, although the task queue backlog shows a downward trend with the increase of the maximum CPU frequency and storage capacity in Fig. 7(b), there will still be some fluctuations similar to the result of network utility.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:05:51 UTC from IEEE Xplore. Restrictions apply.

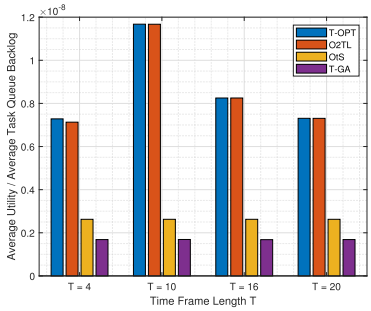


Fig. 8. Average Utility/Average Task Queue Backlog versus Different Time Frame Length T.

5) Comparisons with Benchmarks: To further understand the performance of our online algorithm, we compare the O2TL algorithm with three benchmarks:

- T-time Optimal (T-OPT): The first algorithm is an offline algorithm, which assumes that the MEC system can get future information of next time frame with length T, and then make the decisions to maximize the average network utility and maintain the task queue backlog stable.
- One-time Slot (OtS): This compared algorithm is a special case when Time Frame Length T in our algorithm O2TL is set to 1. This scheme makes service placement decision and task offloading decision simultaneously, as used by [20], [45].
- T-time Greedy Algorithm (T-GA): In this algorithm, when the task computation demands arrive, the MEC system always make task offloading decisions at each time slots and service placement decision at each time frame to maximize the network utility. In addition, surplus task computation demands are randomly migrated to the cloud.

In Fig. 8, we evaluate the performance of utility/queue under different T with benchmarks, where utility/queue means the network utility obtained by the unit task queue backlog. Under different T, the proposed O2TL can achieve a superior solution close to the optimal solution T-OPT. Specifically, the average utility/queue values of O2TL achieve 99.46% of the values of T-OPT, while O2TL is better than OtS and T-GA with average increment ratios of around 227% and 410%, respectively. It can be seen that O2TL can obtain the maximum network utility through the smallest task queue backlog growth, and it is robust under different parameter conditions.

VI. Discussion

In addition to the contributions and findings presented in this paper, there are several promising avenues for future research in the field of service placement and task offloading management in dynamic MEC networks.

Optimization techniques and algorithms. First, further investigation can be conducted to explore advanced optimization techniques and algorithms that can handle the complexities introduced by heterogeneous computation services and varying task demands.

AI-enabled MEC networks. Additionally, the integration of machine learning and artificial intelligence approaches can be explored to develop intelligent decision-making mechanisms for service placement and task offloading, considering real-time network conditions and user preferences.

Distributed computing frameworks. Furthermore, the scalability and robustness of the proposed algorithms can be further improved through the use of distributed computing frameworks and fault-tolerant techniques.

Security and privacy. Lastly, the impact of security and privacy concerns on service placement and task offloading can be thoroughly examined, leading to the development of secure and privacy-preserving solutions.

Overall, these future research directions have the potential to advance the state-of-the-art in dynamic MEC networks and pave the way for more efficient, reliable, and secure mobile edge computing systems.

VII. Conclusion

In this paper, we have addressed the challenge of service placement and task offloading management problem in a dynamic MEC network through an online approach. We formulated the problem within a two-timescale framework, aiming to maximize network utility while ensuring the stability of the entire edge network. To tackle this problem, we proposed a novel online Lyapunov algorithm called O2TL, which leveraged the benefits of the two-timescale framework and employed the Lyapunov optimization technique to make optimal decisions in real-time. We introduced a varying control parameter, $V(t)$, and an Ω-additive approximation to enhance the robustness of the algorithm within an acceptable error margin Ω. Through rigorous theoretical analysis and extensive trace-driven experimental evaluations, we demonstrated the effectiveness of the O2TL algorithm. Our results showed that O2TL outperformed the compared baselines and achieved performance close to the offline optimal solution. Additionally, O2TL achieved a desirable performance-backlog tradeoff with complexity bounds of $[O(1/V), O(V)]$.

References

[1] X. Li, X. Zhang, and T. Huang, "Asynchronous online service placement and task offloading for mobile edge computing," in *Proc. IEEE 18th Annu. Int. Conf. Sensing, Commun., Netw.*, 2021, pp. 1–9.
[2] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Commun. Surveys Tut.*, vol. 19, no. 4, pp. 2322–2358, Fourth Quarter 2017.
[3] P. Mach and Z. Becvar, "Mobile edge computing: A survey on architecture and computation offloading," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 3, pp. 1628–1656, Third Quarter 2017.
[4] N. Abbas, Y. Zhang, A. Taherkordi, and T. Skeie, "Mobile edge computing: A survey," *IEEE Internet Things J.*, vol. 5, no. 1, pp. 450–465, 2017.
[5] C. Feng, P. Han, X. Zhang, B. Yang, Y. Liu, and L. Guo, "Computation offloading in mobile edge computing networks: A survey," *J. Netw. Comput. Appl.*, vol. 202, 2022, Art. no. 103366.
[6] M. Chiosi et al., "Network functions virtualisation: An introduction, benefits, enablers, challenges and call for action," in *Proc. SDN OpenFlow World Congr.*, 2012, pp. 1–16.
[7] N. F. V. ETSI, "Network functions virtualisation (NFV)," *Manage. Orchestration*, vol. 1, 2014, Art. no. V1.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:05:51 UTC from IEEE Xplore. Restrictions apply.

[8] X. Chen, L. Jiao, W. Li, and X. Fu, "Efficient multi-user computation offloading for mobile-edge cloud computing," *IEEE/ACM Trans. Netw.*, vol. 24, no. 5, pp. 2795–2808, Oct. 2015.
[9] Y. Mao, J. Zhang, and K. B. Letaief, "Dynamic computation offloading for mobile-edge computing with energy harvesting devices," *IEEE J. Sel. Areas Commun.*, vol. 34, no. 12, pp. 3590–3605, Dec. 2016.
[10] C. You and K. Huang, "Multiuser resource allocation for mobile-edge computation offloading," in *Proc. IEEE Glob. Commun. Conf.*, 2016, pp. 1–6.
[11] M. Chen and Y. Hao, "Task offloading for mobile edge computing in software defined ultra-dense network," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 3, pp. 587–597, Mar. 2018.
[12] J. L. D. Neto, S.-Y. Yu, D. F. Macedo, J. M. S. Nogueira, R. Langar, and S. Secci, "ULOOF: A user level online offloading framework for mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 17, no. 11, pp. 2660–2674, Nov. 2018.
[13] L. Huang, S. Bi, and Y. J. Zhang, "Deep reinforcement learning for online computation offloading in wireless powered mobile-edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 19, no. 11, pp. 2581–2593, Nov. 2020.
[14] F. Wang, J. Xu, X. Wang, and S. Cui, "Joint offloading and computing optimization in wireless powered mobile-edge computing systems," *IEEE Trans. Wireless Commun.*, vol. 17, no. 3, pp. 1784–1797, Mar. 2017.
[15] J. Zhao, Q. Li, Y. Gong, and K. Zhang, "Computation offloading and resource allocation for cloud assisted mobile edge computing in vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 8, pp. 7944–7956, Aug. 2019.
[16] P. Dai, K. Hu, X. Wu, H. Xing, and Z. Yu, "Asynchronous deep reinforcement learning for data-driven task offloading in mec-empowered vehicular networks," in *Proc. IEEE Conf. Comput. Commun.*, 2021, pp. 1–10.
[17] H. Jiang, X. Dai, Z. Xiao, and A. K. Iyengar, "Joint task offloading and resource allocation for energy-constrained mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 7, pp. 4000–4015, Jul. 2023.
[18] Y. Zhang, F. Tao, Y. Liu, P. Zhang, Y. Cheng, and Y. Zuo, "Long/short-term utility aware optimal selection of manufacturing service composition toward industrial internet platforms," *IEEE Trans. Ind. Inform.*, vol. 15, no. 6, pp. 3712–3722, Jun. 2019.
[19] S. Han et al., "Energy-efficient 5G for a greener future," *Nature Electron.*, vol. 3, no. 4, pp. 182–184, 2020.
[20] J. Xu, L. Chen, and P. Zhou, "Joint service caching and task offloading for mobile edge computing in dense networks," in *Proc. IEEE Conf. Comput. Commun.*, 2018, pp. 207–215.
[21] K. Poularakis, J. Llorca, A. M. Tulino, I. Taylor, and L. Tassiulas, "Joint service placement and request routing in multi-cell mobile edge computing networks," in *Proc. IEEE Conf. Comput. Commun.*, 2019, pp. 10–18.
[22] M. J. Neely, "Stochastic network optimization with application to communication and queueing systems," *Synth. Lectures Commun. Netw.*, vol. 3, no. 1, pp. 1–211, 2010.
[23] T. X. Tran, K. Chan, and D. Pompili, "COSTA: Cost-aware service caching and task offloading assignment in mobile-edge computing," in *Proc. 16th Annu. IEEE Int. Conf. Sens., Commun., Netw.*, 2019, pp. 1–9.
[24] W. Fan et al., "Collaborative service placement, task scheduling, and resource allocation for task offloading with edge-cloud cooperation," in *IEEE Trans. Mobile Comput.*, to be published, doi: 10.1109/TMC.2022.3219261.
[25] A. Naouri, H. Wu, N. A. Nouri, S. Dhelim, and H. Ning, "A novel framework for mobile-edge computing by optimizing task offloading," *IEEE Internet Things J.*, vol. 8, no. 16, pp. 13065–13076, Aug. 2021.
[26] A. Mukherjee, D. De, and D. G. Roy, "A power and latency aware cloudlet selection strategy for multi-cloudlet environment," *IEEE Trans. Cloud Comput.*, vol. 7, no. 1, pp. 141–154, First Quarter 2019.
[27] K. Poularakis, J. Llorca, A. M. Tulino, I. Taylor, and L. Tassiulas, "Service placement and request routing in MEC networks with storage, computation, and communication constraints," *IEEE/ACM Trans. Netw.*, vol. 28, no. 3, pp. 1047–1060, Jun. 2020.
[28] I. Nokia, "Increasing mobile operators value proposition with edge computing," *Tech. Brief*, 2013.
[29] H. Cao, M. Wachowicz, and S. Cha, "Developing an edge analytics platform for analyzing real-time transit data streams," 2017, *arXiv: 1705.08449*.
[30] S. Kekki et al., "MEC in 5G networks," *ETSI White Paper*, vol. 28, pp. 1–28, 2018.
[31] C. Chen, Y. Zeng, H. Li, Y. Liu, and S. Wan, "A multi-hop task offloading decision model in MEC-enabled internet of vehicles," *IEEE Internet Things J.*, vol. 10, no. 4, pp. 3215–3230, Feb. 2023.
[32] I.-H. Hou, T. Zhao, S. Wang, and K. Chan, "Asymptotically optimal algorithm for online reconfiguration of edge-clouds," in *Proc. 17th ACM Int. Symp. Mobile Ad Hoc Netw. Comput.*, 2016, pp. 291–300.
[33] Q. Xie, Q. Wang, N. Yu, H. Huang, and X. Jia, "Dynamic service caching in mobile edge networks," in *Proc. IEEE 15th Int. Conf. Mobile Ad Hoc Sensor Syst.*, 2018, pp. 73–79.
[34] N. Yu, Q. Xie, Q. Wang, H. Du, H. Huang, and X. Jia, "Collaborative service placement for mobile edge computing applications," in *Proc. IEEE Glob. Commun. Conf.*, 2018, pp. 1–6.
[35] L. Chen, C. Shen, P. Zhou, and J. Xu, "Collaborative service placement for edge computing in dense small cell networks," *IEEE Trans. Mobile Comput.*, vol. 20, no. 2, pp. 377–390, Feb. 2019.
[36] S. Pasteris, S. Wang, M. Herbster, and T. He, "Service placement with provable guarantees in heterogeneous edge computing systems," in *Proc. IEEE Conf. Comput. Commun.*, 2019, pp. 514–522.
[37] Y. Mao, J. Zhang, S. Song, and K. B. Letaief, "Stochastic joint radio and computational resource management for multi-user mobile-edge computing systems," *IEEE Trans. Wireless Commun.*, vol. 16, no. 9, pp. 5994–6009, Sep. 2017.
[38] J. Ren et al., "An efficient two-layer task offloading scheme for MEC system with multiple services providers," in *Proc. IEEE Conf. Comput. Commun.*, 2022, pp. 1519–1528.
[39] I. Gurobi Optimization, "Gurobi optimizer reference manual," 2018. [Online]. Available: http://www.gurobi.com
[40] A. Kiani and N. Ansari, "Toward hierarchical mobile edge computing: An auction-based profit maximization approach," *IEEE Internet Things J.*, vol. 4, no. 6, pp. 2082–2091, Dec. 2017.
[41] G. Zhang, W. Zhang, Y. Cao, D. Li, and L. Wang, "Energy-delay tradeoff for dynamic offloading in mobile-edge computing system with energy harvesting devices," *IEEE Trans. Ind. Inform.*, vol. 14, no. 10, pp. 4642–4655, Oct. 2018.
[42] B. Gao, Z. Zhou, F. Liu, and F. Xu, "Winning at the starting line: Joint network selection and service placement for mobile edge computing," in *Proc. IEEE Conf. Comput. Commun.*, 2019, pp. 1459–1467.
[43] C. Reiss, J. Wilkes, and J. L. Hellerstein, "Google cluster-usage traces: format+ schema," *Google Inc., White Paper*, vol. 1, pp. 1–14, 2011.
[44] S. M. Ross, *Introduction to Probability Models*. New York, NY, USA: Academic, 2014.
[45] Q. Zhang, L. Gui, F. Hou, J. Chen, S. Zhu, and F. Tian, "Dynamic task offloading and resource allocation for mobile edge computing in dense cloud RAN," *IEEE Internet Things J.*, vol. 7, no. 4, pp. 3282–3299, Apr. 2020.



Xin Li received the master's degree in computer science and engineering from the South China University of Technology, in 2021. He is currently working toward the PhD degree with Rutgers University. His research interests include mobile crowdsensing and mobile computing.



Xinglin Zhang (Member, IEEE) received the BE degree from the School of Software, Sun Yat-sen University, in 2010 and the PhD degree from the Department of Computer Science and Engineering, Hong Kong University of Science and Technology, in 2014. He is currently a professor with the South China University of Technology. His research interests include mobile crowdsensing, edge computing, and mobile computing. He is a member of ACM.



Tiansheng Huang received the BE degree in computer science and engineering from the South China University of Technology, in 2019. He is currently working toward the master's degree with the South China University of Technology. His research interests include fog computing and optimization theory.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:05:51 UTC from IEEE Xplore. Restrictions apply.